



Project report On

**NyayaSahayi: A Unified AI-Driven System for
Integrated Legal Workflow**

*Submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Computer Science and Engineering

By

Ahmed Abdul Majeed (U2203019)

Alan J Arackal (U2203021)

Alsha Rose George (U2203032)

Arya Vinod (U2203053)

Under the guidance of

Ms. Janani K

**Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)**

Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

*This is to certify that the project report entitled "**NyayaSahayi: A Unified AI-Driven System for Integrated Legal Workflow**" is a bonafide record of the work done by **Ahmed Abdul Majeed(U2203019)**, **Alan J Arackal (U2203021)**, **Alsha Rose George (U2203032)**, **Arya Vinod (U2203053)**, submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Project Guide
Ms. Janani K
Assistant Professor
Dept. of CSE
RSET

Project Coordinator
Dr. Jisha G
Associate Professor
Dept. of CSE
RSET

Head of Department
Dr. Preetha K G
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Fr. Dr. Jaison Paul Mulerrickal CMI**, Principal of RSET, and **Dr. Preetha K G**, Head of the Department of Computer Science and Engineering for providing us with the opportunity to undertake our project, "NyayaSahayi : A Legal Assistant".

We are highly indebted to our project coordinators, **Dr. Jisha G**, Associate Professor, Department of Computer Science and Engineering, and **Dr. Preetha K G**, Associate Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Janani K**, for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Ahmed Abdul Majeed

Alan J Arackal

Alsha Rose George

Arya Vinod

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards **Fr. Dr. Jaison Paul Mulerrickal CMI**, Principal of RSET, and **Dr. Preetha K G**, Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my project, "NyayaSahayi : A Legal Assistant".

I am highly indebted to my project coordinators, **Dr. Jisha G**, Associate Professor, Department of Computer Science and Engineering, and **Dr. Preetha K G**, Associate Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Ms. Janani K**, for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Ahmed Abdul Majeed

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards **Fr. Dr. Jaison Paul Mulerrickal CMI**, Principal of RSET, and **Dr. Preetha K G**, Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my project, "NyayaSahayi : A Legal Assistant".

I am highly indebted to my project coordinators, **Dr. Jisha G**, Associate Professor, Department of Computer Science and Engineering, and **Dr. Preetha K G**, Associate Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Ms. Janani K**, for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Alan J Arackal

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards **Fr. Dr. Jaison Paul Mulerrickal CMI**, Principal of RSET, and **Dr. Preetha K G**, Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my project, "NyayaSahayi : A Legal Assistant".

I am highly indebted to my project coordinators, **Dr. Jisha G**, Associate Professor, Department of Computer Science and Engineering, and **Dr. Preetha K G**, Associate Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Ms. Janani K**, for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Alsha Rose George

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards **Fr. Dr. Jaison Paul Mulerrickal CMI**, Principal of RSET, and **Dr. Preetha K G**, Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my project, "NyayaSahayi : A Legal Assistant".

I am highly indebted to my project coordinators, **Dr. Jisha G**, Associate Professor, Department of Computer Science and Engineering, and **Dr. Preetha K G**, Associate Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Ms. Janani K**, for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Arya Vinod

Abstract

The Indian legal ecosystem, particularly at the practitioner level, is characterized by fragmented workflows, high documentation overhead, and communication gaps between lawyers and their clients. This project presents NyayaSahayi, an integrated AI-powered Legal Assistant platform designed to address these challenges through four interconnected modules. The first module, an Intelligent Client Communication System, provides real-time case context popups during incoming calls and deploys an automated Malayalam voice-chatbot via SMS for missed calls to ensure accessible client support. The second module is an Automated Legal Document Generator that significantly reduces manual effort by auto-populating templates and enabling bulk document creation from spreadsheet data. The third module features a Legal Knowledge Chatbot built on a hybrid GraphRAG architecture, combining knowledge graph construction with retrieval-augmented generation to provide nuanced, citation-aware answers grounded in the BNS, BNSS, and BSA. Finally, a Diary-to-Calendar Intelligence System digitizes physical legal diaries using OCR, cross-references extracted hearing dates with official court records, and pushes verified schedules to a centralized calendar with automated reminders. During system evaluation, the platform demonstrated highly responsive performance, including sub-200ms latency for real-time call popups. Furthermore, the GraphRAG architecture successfully mitigated large language model hallucinations by strictly constraining generated responses to verified statutory context. Together, these modules form a cohesive platform that modernizes legal practice management, reduces administrative burdens, and improves client communication with particular sensitivity to the linguistic and procedural context of Kerala’s legal practitioners.

Contents

Acknowledgment	i
Acknowledgment	i
Acknowledgment	i
Acknowledgment	i
Acknowledgment	i
Abstract	ii
List of Abbreviations	vi
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background	1
1.2 Problem Definition	2
1.3 Scope and Motivation	3
1.4 Objectives	3
1.5 Challenges	4
1.6 Assumptions	4
1.7 Societal / Industrial Relevance	5
1.8 Organization of the Report	5
1.9 Summary of Chapter	5
2 Literature Survey	6
2.1 Introduction	6

2.2	Survey Papers	6
2.2.1	HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications [1]	6
2.2.2	An LLM-assisted ETL pipeline to build a high-quality knowledge graph of the Italian legislation [2]	9
2.2.3	Toward Low-Latency End-to-End Voice Agents for Telecommuni- cations Using Streaming ASR, Quantized LLMs, and Real-Time TTS.[3]	11
2.2.4	Legal Document Generation: Harnessing Large Language Models for Diverse Legal Workflows [4]	13
2.3	Comparison of Papers	16
2.4	Gaps identified	16
2.5	Summary	17
3	System Design	18
3.1	Overall Architecture	18
3.2	Component Design	19
3.2.1	Client Communication Module	19
3.2.2	Document Generation Module	20
3.2.3	RAG-Based Legal Chatbot Module	21
3.2.4	Diary to Calendar Module	22
3.3	Tools and Technologies	24
3.4	Work Breakdown and Responsibilities	25
3.5	Key Deliverables	25
3.6	Project Timeline	26
3.7	Summary of Chapter	26
4	Results and Discussion	27
4.1	Introduction	27
4.2	GraphRAG Legal Research Chatbot	27
4.3	Real-Time Call Popup Module	28
4.4	Document Generation and Automation Module	31
4.5	Automated Docketing and Calendar Synchronisation	33

4.6	Discussion	33
4.6.1	Strengths of the Unified Architecture	33
4.6.2	Limitations	33
4.7	Summary	34
5	Conclusion and Future Scope	35
5.1	Conclusion	35
5.2	Future Scope	36
	References	37
	Appendix A: Presentation	39
	Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes	65
	Appendix C: CO-PO-PSO Mapping	69

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BNS	Bharatiya Nyaya Sanhita
BNSS	Bharatiya Nagarik Suraksha Sanhita
BSA	Bharatiya Sakshya Adhiniyam
CrPC	Code of Criminal Procedure
FCM	Firebase Cloud Messaging
IPC	Indian Penal Code
KG	Knowledge Graph
LLM	Large Language Model
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
TTS	Text-to-Speech
UI	User Interface

List of Figures

3.1	Full System Architecture Design	19
3.2	Component Design of Real-Time Context-Aware Call Popup System	20
3.3	Component Design of Client Chat Interface	20
3.4	Component Design of Document Generation Module	21
3.5	Component Design of GraphRAG based Chatbot	22
3.6	Component Design of Diary to Calendar	23
4.1	GraphRAG based Chatbot for Lawyers	29
4.2	Knowledge Graph	29
4.3	Pop Up During an Incoming Call	30
4.4	SMS with Unique URL and Client Chatbot Interface	30
4.5	Bulk Document Generation	31
4.6	Generating Legal Documents from Templates	32
4.7	Mobile UI	32

List of Tables

2.1	Summary of literature papers	16
3.1	Project Timeline	26
5.1	CO-PO-PSO Mapping	70

Chapter 1

Introduction

The legal profession is one of the most information-intensive fields, requiring practitioners to manage voluminous case records, strict court deadlines, and continuous client communication simultaneously. In India, this challenge is compounded by an overburdened judiciary and a practitioner community that still relies heavily on manual, paper-based processes.

This project proposes an integrated AI-powered Legal Assistant Platform addressing these pain points through four modules: an intelligent client communication system, an automated document generation engine, a GraphRAG-based legal knowledge chatbot, and a diary-to-calendar intelligence pipeline. Designed for Kerala’s legal practitioners, the platform incorporates Malayalam language support to ensure accessibility at the grass-roots level.

1.1 Background

India enacted three landmark criminal law reforms in 2023 the Bharatiya Nyaya Sanhita (BNS) [5], the Bharatiya Nagarik Suraksha Sanhita (BNSS) [6], and the Bharatiya Sakshya Adhinyam (BSA) [7] replacing the Indian Penal Code, Code of Criminal Procedure, and the Indian Evidence Act respectively. These reforms introduced significant changes in legal provisions and terminologies, creating an immediate demand for accessible legal knowledge tools.

Simultaneously, the eCourts Mission Mode Project has digitized case records and hearing schedules for many courts, making this data programmatically accessible [8]. However, discrepancies between online records and ground realities persist, especially in lower courts and for recently transferred or freshly filed cases.

To bridge this gap between complex, evolving legal data and daily practitioner workflows, recent advances in artificial intelligence offer a highly effective solution. Specifically, breakthroughs in natural language processing (NLP), Large Language Models (LLMs), Retrieval-Augmented Generation (RAG)[9], and knowledge graph construction [10] now make it feasible to build domain-specific legal assistants. These technologies allow systems to provide contextually accurate, citation-backed responses, making the practical deployment of a comprehensive, AI-powered legal practice platform achievable

1.2 Problem Definition

Despite the growing availability of digital tools, most solo practitioners and small to mid-sized law practices in Kerala remain largely unautomated. This lack of digital integration creates significant operational bottlenecks, particularly in client communication and administrative workflows. Lawyers frequently miss client calls during active court hours, and without a structured fallback system, this inevitably erodes client confidence and results in repetitive, unnecessary follow-ups. Furthermore, routine legal document preparation remains a time-consuming and error-prone manual process, lacking automated or bulk generation capabilities for high-volume filings such as bail applications.

Beyond administrative overload, the recent national transition to the Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS), and Bharatiya Sakshya Adhiniyam (BSA) has placed a massive cognitive strain on practitioners, who must rapidly familiarize themselves with an overhauled legal framework. Compounding this is the inefficiency of current scheduling methods; official court websites under the eCourts portal often experience delays in reflecting rescheduled dates, and there is no automated mechanism to reconcile a lawyer’s handwritten physical diary entries with scraped digital court records. Finally, the fact that most existing legal technology operates exclusively in English creates a severe usability barrier for Malayalam-speaking clients in Kerala, limiting their access to critical case updates and legal resources

1.3 Scope and Motivation

This project is scoped to solo practitioners and small law firms in Kerala, with the legal knowledge base aligned to Indian national criminal law under the Bharatiya Nyaya Sanhita (BNS)[5], Bharatiya Nagarik Suraksha Sanhita (BNSS)[6], and Bharatiya Sakshya Adhiniyam (BSA)[7]. The platform encompasses four core modules: client call management with SMS fallback and a Malayalam voice bot, template-based and bulk document generation, a GraphRAG-powered legal chatbot [9], and a diary-to-calendar pipeline with court data reconciliation. Architecturally, the platform is designed as a mobile-first application specifically targeting Android to leverage native telephony integration supported by a cloud-based backend, requiring no specialized hardware beyond a standard smartphone.

The motivation for this project is grounded in a commitment to access to justice and technological equity. Legal services function as a critical public good, and improvements in practitioner efficiency directly translate to better representation for citizens in rural and semi-urban areas. By automating administrative overhead, practitioners can dedicate more time to substantive legal strategy. Additionally, the inclusion of Malayalam voice support ensures the technology meets grassroots users in their native language rather than imposing an additional digital or literacy barrier.

1.4 Objectives

- To develop an intelligent call management module that identifies registered clients, displays relevant case information during calls, and dispatches automated SMS with a bot link upon missed calls.
- To build a conversational client-facing chatbot that responds to case-related queries in text and Malayalam voice, accessible via a shareable link.
- To design a document generation system that auto-populates templates from a client database and supports bulk document creation from Excel inputs.
- To implement a legal knowledge chatbot grounded in BNS, BNSS, and BSA [5, 6, 7] using GraphRAG [1] combining knowledge graph traversal [10] with dense retrieval

[9] for accurate, context-aware responses.

- To develop a diary intelligence module that uses OCR to extract hearing data from physical diary scans, cross-references it with official court data [8], and populates a unified calendar with automated reminders.
- To integrate all modules into a cohesive platform suitable for solo practitioners and small law firms.

1.5 Challenges

Achieving reliable OCR on handwritten legal diaries is technically demanding, as entries contain mixed English and Malayalam scripts with abbreviations and informal shorthand. Standard OCR engines perform poorly on such inputs, necessitating specialised handwriting recognition models with post-processing heuristics.

Building the legal knowledge graph [10] from BNS, BNSS, and BSA [5, 6, 7] requires accurately capturing dense cross-references, definitional dependencies, and procedural chains as graph edges. The diary-to-calendar reconciliation additionally requires a robust confidence-scoring mechanism to resolve conflicts between diary entries and scraped court records without human intervention for every case.

1.6 Assumptions

- Lawyers are willing to maintain a structured digital client database containing at least the client’s name, phone number, and primary case reference
- Physical diaries submitted for digitization are legible enough for OCR processing with reasonable character clarity.
- Clients have access to a smartphone capable of opening a web-based chatbot link received via SMS.
- The deployment environment has stable internet connectivity for scraping, API calls, and LLM inference.

1.7 Societal / Industrial Relevance

This platform improves access to justice by freeing lawyers from routine administrative tasks, enabling better representation for citizens in rural and semi-urban areas. The Malayalam voice interface breaks the language barrier that has historically excluded regional litigants from digital legal tools. The legal knowledge chatbot promotes broader legal literacy by making BNS, BNSS, and BSA provisions [5, 6, 7] accessible in plain, queryable language, while automated hearing reminders contribute to faster case resolution in an already overburdened judiciary.

From an industrial perspective, the platform addresses a significant gap in India’s emerging legal technology market. The GraphRAG architecture [1] advances the state of practice for AI deployment in high-stakes professional contexts, offering a replicable blueprint for other regulated industries such as healthcare and taxation. The bulk document generation and diary digitisation modules demonstrate how AI can unlock efficiency in paper-based professional workflows well beyond the legal sector.

1.8 Organization of the Report

Chapter 2 presents the literature survey covering related work in RAG systems, knowledge graphs, voice agents, and legal document generation. Chapter 3 describes the system design and architecture of all four modules. Chapter 4 presents the results and evaluation. Chapter 5 concludes with a summary and future scope.

1.9 Summary of Chapter

This chapter introduces NyayaSahayi, an AI-driven legal platform designed to automate administrative workflows for small practices in Kerala. It tackles key challenges in client communication, repetitive document drafting, and navigating the new criminal laws (BNS, BNSS, and BSA) through four integrated modules: an intelligent call manager, an automated document generator, a GraphRAG legal chatbot, and an OCR-based diary-to-calendar pipeline.

Chapter 2

Literature Survey

2.1 Introduction

The legal profession is increasingly adopting technology to enhance efficiency, reduce administrative overhead, and improve client services. This project proposes the development of a comprehensive AI-powered assistant for lawyers, designed to automate key tasks and streamline workflows. The system integrates several functionalities, including an intelligent call management system with client detail pop-ups, a legal query chatbot, automated scheduling from diary photos, proactive case tracking from court websites, and automated document generation.

To ensure a robust design, this literature survey will review existing research and established technologies in the core areas that enable these features. We will examine state-of-the-art methodologies and frameworks in Natural Language Processing (NLP) for the chatbot and call handler, Computer Vision (OCR) for diary scanning, and automated information retrieval for website data extraction. This review will identify the most effective approaches to leverage for the successful implementation of this multifaceted legal assistant.

2.2 Survey Papers

2.2.1 HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications [1]

This paper introduces the Hybrid Parameter-Adaptive RAG (HyPA-RAG) system [1], designed to improve the performance of Large Language Models in the legal and policy domain. To overcome common issues like retrieval errors and high operational costs,

HyPA-RAG uses a novel architecture that combines a hybrid retrieval method (dense, sparse, and knowledge graph) with a query complexity classifier. This classifier dynamically adjusts system parameters based on the user’s query, optimizing both accuracy and efficiency. The system was tested on NYC Local Law 144 and demonstrated enhanced retrieval accuracy and response fidelity.

Objective:

The primary objective is to address the challenges that standard Retrieval-Augmented Generation (RAG) systems face in the complex legal domain, such as retrieval errors, poor context integration, and high costs. The goal is to create a robust and adaptable solution that enhances retrieval accuracy, response fidelity, and contextual precision for high-stakes legal and policy applications.

Key Features:

- **Hybrid Retrieval:** The system integrates three different retrieval methods dense (vector), sparse (BM25), and knowledge graph retrieval to improve the accuracy and comprehensiveness of the retrieved information.
- **Adaptive Parameter Selection:** A core feature is the query complexity classifier, which analyzes user queries to dynamically adjust retrieval parameters like ‘top-k’ values and the number of query rewrites. This makes the system more efficient and tailored to the specific query.
- **Knowledge Graph (KG) Integration:** A knowledge graph is constructed from the legal text to capture complex relationships and entities that simple text retrieval might miss.
- **Comprehensive Evaluation Framework:** The authors developed a custom ”gold standard” dataset with various question types and used specialized RAG metrics like Faithfulness, Answer Relevancy, and Context Precision for a thorough evaluation.

Advantages:

- **Enhanced Retrieval Accuracy and Response Fidelity:** HyPA-RAG integrates a query complexity classifier and a hybrid retrieval approach (dense, sparse, and

knowledge graph methods), which collectively enhance retrieval accuracy and the fidelity of responses, especially in complex legal contexts.

- **Adaptive Parameter Tuning:** The system uses a query complexity classifier for adaptive parameter tuning, allowing it to dynamically adjust parameters like the number of query rewrites (Q), top-k value (k), maximum keywords (K), and maximum knowledge sequence length (S) based on query complexity. This adaptivity helps reduce token usage and improve accuracy.
- **Addresses LLM Limitations in High-Stakes Domains:** HyPA-RAG directly tackles common limitations of LLMs in legal and policy applications, such as outdated knowledge, hallucinations (factually incorrect outputs), and poor reasoning in complex contexts, by incorporating external knowledge and robust retrieval techniques.
- **Robust Evaluation Framework:** The research includes a comprehensive evaluation framework with tailored question types (simple, complex, situational, comparative, etc.) and RAG-specific metrics (Faithfulness, Answer Relevancy, Context Precision, Context Recall, and an adapted Correctness metric). This rigorous evaluation demonstrates the system’s effectiveness and provides insights into its performance.

Disadvantages and Limitations:

- **Evaluation Limitations:** The correctness evaluation relied on a single evaluator, which could be a limitation; using a larger group of human evaluators would improve reliability.
- **Generalizability Concerns:** The query classifier was trained on synthetically generated data, which, despite noise injection, might contain biases from the LLM used to create it, potentially limiting how well it generalizes to real-world user queries.
- **Knowledge Graph Complexity:** While integrating a knowledge graph helps maintain correctness, it adds complexity to the system and can sometimes lower the overall quality of the response.

- **Parameter Mapping Validation:** The mappings used to adapt parameters based on query complexity were not rigorously validated quantitatively, suggesting that further evaluation could lead to better performance.

Relevance to Project:

The HyPA-RAG architecture provides a sophisticated blueprint for an AI legal assistant. Its hybrid retrieval system, especially the integration of a knowledge graph, is directly applicable for accurately querying legal documents, overcoming the limitations of simple text retrieval. Furthermore, the adaptive parameter selection based on query complexity is a key innovation that could make the assistant more efficient and cost-effective. The paper’s detailed evaluation framework also offers a robust methodology for testing the final system’s reliability.

2.2.2 An LLM-assisted ETL pipeline to build a high-quality knowledge graph of the Italian legislation [2]

This paper details an ETL pipeline that builds a high-quality knowledge graph of Italian legislation. It uses the Akoma Ntoso (AKN) XML standard for reliable data mapping and enhances the graph using fine-tuned LLMs like BERT and Mistral-7B to add missing information such as topics, titles, and ministry domains. The result is a consistent and queryable graph that accurately represents the evolution of laws over time.

Objective:

The main objective is to design and implement an end-to-end pipeline that transforms Italian legislation, published in the AKN format, into a high-quality, queryable Property Graph. The goal is to create a robust and enriched data model that supports in-depth analysis of legislative data, which is achieved by combining a solid XML-to-graph mapping with LLM-based data enhancement.

Key Features:

- **Property Graph Model:** Models legislation using a flexible property graph where nodes and edges have attributes.

- **AKN Standard Mapping:** Reliably maps legal documents to graph objects using the international Akoma Ntoso XML standard.
- **LLM Enrichment:** Uses fine-tuned LLMs (BERT, Mistral-7B) to add missing properties like domains, titles, and topics to the graph.
- **Temporal Modeling:** Tracks legal changes over time by storing amendments and repeals as edge properties, allowing for time-specific queries
- **Error Correction:** Employs graph queries to automatically detect and correct data inconsistencies in the graph.

Advantages:

- **High Quality:** The use of the structured AKN standard and graph-based error correction results in a highly accurate and consistent knowledge graph.
- **Enriched Data:** LLMs significantly improve the graph's completeness by adding valuable metadata not present in the source files, enabling advanced analysis.
- **Efficient Updates:** The system is designed for efficient daily updates, ensuring the entire graph remains current by processing only new laws.
- **Sustainable & Reproducible:** The use of lightweight open source LLMs makes the pipeline more sustainable and easier to replicate.

Disadvantages and Limitations:

- **Format Dependent:** The process relies heavily on structured XML data (like Akoma Ntoso) and is not easily adaptable to unstructured formats like PDF.
- **Source Data Quality:** The final accuracy is limited by errors in the original source documents, such as missing tags that the pipeline cannot fix.
- **Complex Setup:** The LLM components require specialized fine-tuning with custom datasets, which complicates the initial setup.

Relevance to Project:

This paper is highly relevant as it offers a clear methodological blueprint to construct a knowledge graph from legal documents, which is a core feature of the proposed AI assistant. The paper’s use of LLMs to enrich the graph by extracting topics is directly applicable to building the legal query chatbot, as an enriched graph enables more effective retrieval of relevant laws and articles. Furthermore, the method detailed for tracking legal changes is crucial; the assistant must provide current information, and the paper’s graph model efficiently handles amendments and repeals to ensure the data remains up-to-date.

2.2.3 Toward Low-Latency End-to-End Voice Agents for Telecommunications Using Streaming ASR, Quantized LLMs, and Real-Time TTS.[3]

This paper introduces a low-latency, end-to-end voice agent designed for real-time telecommunications applications . The system integrates a streaming Automatic Speech Recognition (ASR), a 4-bit quantized Large Language Model (LLM), Retrieval-Augmented Generation (RAG) for contextual responses, and a real-time Text-to-Speech (TTS) module. By using techniques like sentence-level streaming, concurrent processing, and binary serialization, the pipeline achieves an average response time of under one second while maintaining high semantic relevance in its answers.

Objective:

The primary objective is to develop a complete voice-to-voice pipeline purpose-built for interactive telecommunication scenarios that minimizes end-to-end latency. The goal is to create a responsive, knowledge-grounded conversational agent by effectively integrating and optimizing streaming ASR, a quantized LLM, RAG, and real-time TTS components within a multi-threaded architecture.

Key Features:

- **Integrated Voice Pipeline:** Combines streaming ASR, a quantized LLM, RAG, and real-time TTS for seamless voice-to-voice interaction.
- **Low-Latency Optimizations:** Achieves sub-second responses using **sentence-level streaming**, concurrent multi-threaded processing, and binary serialization.
- **Efficient LLM:** Utilizes a 4-bit quantized LLM to reduce memory usage and speed up inference without significant quality loss.

- **Contextual RAG:** Employs Retrieval-Augmented Generation (RAG) with FAISS similarity search to ground the LLM’s responses in factual, domain-specific documents.

Advantages:

- **Low Latency:** The system achieves an average total latency of 0.94 seconds, making it suitable for interactive conversations. This is accomplished through sentence-level streaming and concurrent execution of modules.
- **Resource Efficiency:** The use of a 4-bit quantized LLM significantly reduces the GPU memory footprint and speeds up inference without a major loss in generation quality.
- **Contextual Accuracy:** By integrating Retrieval-Augmented Generation (RAG), the agent can retrieve information from relevant documents to provide accurate and knowledge-grounded responses.
- **High Throughput:** The components operate at high speeds suitable for real-time applications, with the ASR processing at an average of 394 tokens/sec and the LLM generating 80 tokens/sec.

Disadvantages and Limitations:

- **Dependency on ASR Accuracy:** The system’s overall performance is limited by the accuracy of the Automatic Speech Recognition (ASR) output. Transcription inaccuracies, particularly with abbreviations or domain-specific terms, can degrade the performance of the RAG system’s similarity search.
- **Latency Variability:** While the mean latency is low, the worst-case total pipeline latency can be significantly higher, reaching over 3 seconds. This variability is likely caused by fluctuations in GPU processing.
- **Domain-Specific Focus:** The agent was evaluated on a custom dataset created for the telecommunications domain. Its performance in other specialized domains is not guaranteed and would require adaptation.

Relevance to Project:

This paper is highly relevant to the development of an AI assistant for lawyers. The architecture for a low-latency voice agent provides a direct blueprint for the feature that answers calls from clients, ensuring conversations are smooth and natural. More importantly, the use of Retrieval-Augmented Generation (RAG) is a critical technique for the legal query chatbot. By implementing RAG, the chatbot can retrieve precise information from the project’s own knowledge graph of legal documents, ensuring its answers are not just conversational but also factually accurate and grounded in specific legal texts, which is essential for the legal domain.

2.2.4 Legal Document Generation: Harnessing Large Language Models for Diverse Legal Workflows [4]

This paper presents an AI-powered framework for automating the generation of diverse legal documents such as Statements of Work (SoW), Procurement agreements, Treasury reports, Accounts Payable documents, and IAUD policies. The proposed methodology employs a multi-agent system integrated with iterative prompt engineering and Large Language Models (LLMs) to generate structured, compliant legal documents with minimal user intervention. The framework is validated through real-world deployment within the author’s organization, demonstrating measurable gains in speed, accuracy, and operational efficiency.

Objective:

The primary objective is to bridge the existing gap between LLM capabilities and automated legal document generation in professional workflows. The system aims to drastically reduce the time and manual effort involved in creating legal documents from scratch, eliminate inconsistencies and compliance errors common in manually authored documents, and generate fully structured, template-compliant documents based on minimal user input. An additional goal is to incorporate auto-generated process flowcharts into process documents, enabling end-to-end streamlining of legal document workflows. The research also contributes to the broader discourse on LLM adoption in legal domains, demonstrating practical applicability through empirical validation.

Key Features:

- **Multi-Agent Architecture:** The framework employs specialized agents for dis-

tinct tasks like an Information Extractor and Processor for parsing user inputs, a Semantic Identifier for understanding contextual meaning, a Template Analyser for mapping content to the correct document structure, and a Content Generator for producing the final output using the LLM.

- **Prompt Engineering:** Custom-crafted prompts incorporating legal terminology, document structure guidelines, and user inputs instruct the LLM backend. Section-specific prompt refinement is used iteratively to optimize output quality for each part of a document.
- **Template-Driven Compliance:** Standard templates were developed in collaboration with in-house legal experts for each document type, ensuring that all generated outputs conform to company-specific and legal standards.
- **Web-Based Deployment:** A simple UI allows users to provide document-specific inputs and receive a complete, editable Word document in return, requiring minimal training.

Advantages:

- **Time Reduction:** The system reduces document creation time by approximately 99%.
- **Improved Accuracy and Compliance:** By adhering to predefined templates, the system minimizes errors and inconsistencies.
- **Enhanced Turnaround Efficiency:** Overall turnaround time including document creation and error correction, improved by an average of 89.66% across all document types.

Disadvantages and Limitations:

- **English Language Dependency:** The system was developed primarily for English-speaking users. Non-English professionals must translate output manually, limiting global accessibility.
- **Template Specificity:** The framework requires tailor-stitched prompts and templates for each document type. Introducing a new document category requires code-level modifications.

- **Document-Specific Knowledge Required:** Generating compliant output for a new document type demands prior knowledge of its structure and legal requirements.

Relevance to Project:

This paper propose a multi-agent LLM framework with iterative prompt refinement to automate document generation in a structure-compliant manner. This directly aligns with NyayaSahayi, where Google Gemini analyses imported legal documents and identifies variable fields through a structured multi-stage prompt, mirroring the paper’s semantic pipeline. Unlike the paper’s fully automated approach, NyayaSahayi adopts a hybrid model where AI handles field detection while the lawyer retains final editorial control, better suited to Indian legal compliance. Both works also target high-volume workflows, NyayaSahayi’s bulk engine generates multiple client-specific PDFs from a single spreadsheet upload.

2.3 Comparison of Papers

Paper Title	Methodology	Strengths	Limitations
HyPA-RAG System	Hybrid dense/sparse/KG retrieval with adaptive query tuning.	Improved retrieval accuracy; adaptive cost-efficiency; supports complex queries.	Trained on synthetic data; single evaluator; KG adds system complexity.
LLM-assisted ETL Pipeline for Italian Legislation	ETL converts Akoma Ntoso XML into a property graph; fine-tuned BERT and Mistral-7B enrich missing domains/topics.	Structured and consistent data; supports advanced queries; efficient for daily updates.	Depends on AKN format; errors in source docs affect output; complex LLM fine-tuning required.
Low-Latency End-to-End Voice Agents	Streaming ASR, 4-bit LLM, RAG, and TTS integrated for real-time response.	Low latency ; efficient quantized model; accurate contextual replies.	ASR errors impact quality; occasional latency spikes; domain-specific model.
Legal Document Generation Using LLMs	Multi-agent system with prompt engineering and LLMs.	99% faster document creation; 89.66% turnaround improvement; scalable deployment.	English-only;Template Specificity;Document-Specific Knowledge Required.

Table 2.1: Summary of literature papers

2.4 Gaps identified

Based on the literature survey, several key research gaps emerge in the development of comprehensive AI legal assistants. A foundational challenge is the need for robust knowledge extraction from unstructured documents like PDFs, as current methods often rely on standardized formats. Following this, a gap exists in advanced query understanding; while some systems can gauge query complexity, they may not generalize well to the

nuances of real-world legal questions. This leads to a further challenge in the optimal integration of hybrid retrieval methods. Although combining dense, sparse, and knowledge graph retrieval is beneficial, the best strategy for fusing these sources to avoid increased complexity or reduced response quality remains an open question.

Furthermore, current document generation approaches are often overly simplistic, relying on template-based models that are prone to overfitting and lack the flexibility required for complex legal drafting. The most significant overarching gap is the absence of a unified, multifunctional architecture. The literature presents specialized solutions for individual tasks like voice interaction, knowledge graph creation, or question-answering, but there is a clear need to integrate these disparate functions into a single, seamless tool for legal professionals.

2.5 Summary

The literature review shows that AI can effectively create knowledge graphs from structured legal texts , enable low-latency voice agents for real-time communication , and improve question-answering with advanced hybrid retrieval systems. However, these approaches face significant limitations. Knowledge extraction struggles with the unstructured PDFs common in legal practice , and document generation tools are often restricted to simple, template-based outputs that can suffer from model overfitting. A major gap identified is the absence of a unified architecture that integrates these distinct capabilities into a single, comprehensive AI assistant for legal professionals.

Chapter 3

System Design

The system design chapter presents the architectural foundation of the AI-powered Legal Assistant Platform, translating four problem areas client communication, document generation, legal knowledge access, and hearing schedule management into an interconnected technical system for legal practitioners in Kerala. The platform is a coordinated ensemble of intelligent modules, each addressing a specific workflow bottleneck while sharing a common client database and communication backbone.

3.1 Overall Architecture

The platform is organised into two primary zones, a cloud-based backend and a lawyer’s local device environment connected through event-driven data flows that automate core legal practice workflows.

The backend hosts four major modules: the Voice Agent, the Document Generation Module, the Diary to Calendar converter, and the RAG-Based Chatbot [9]. The Voice Agent handles outbound SMS and voice interactions for unanswered calls. The RAG-Based Chatbot is powered by a dual-database retrieval layer comprising a Vector Database and a Graph Database [10], both built from the BNS [5], BNSS [6], and BSA [7].

On the lawyer’s device, an incoming call triggers the Call Monitor, which checks the local database and renders a Display Popup with relevant case details. The lawyer’s local database remains continuously synchronised with the backend server via Firebase Cloud Messaging [11], ensuring all responses reflect the most current case data [8].

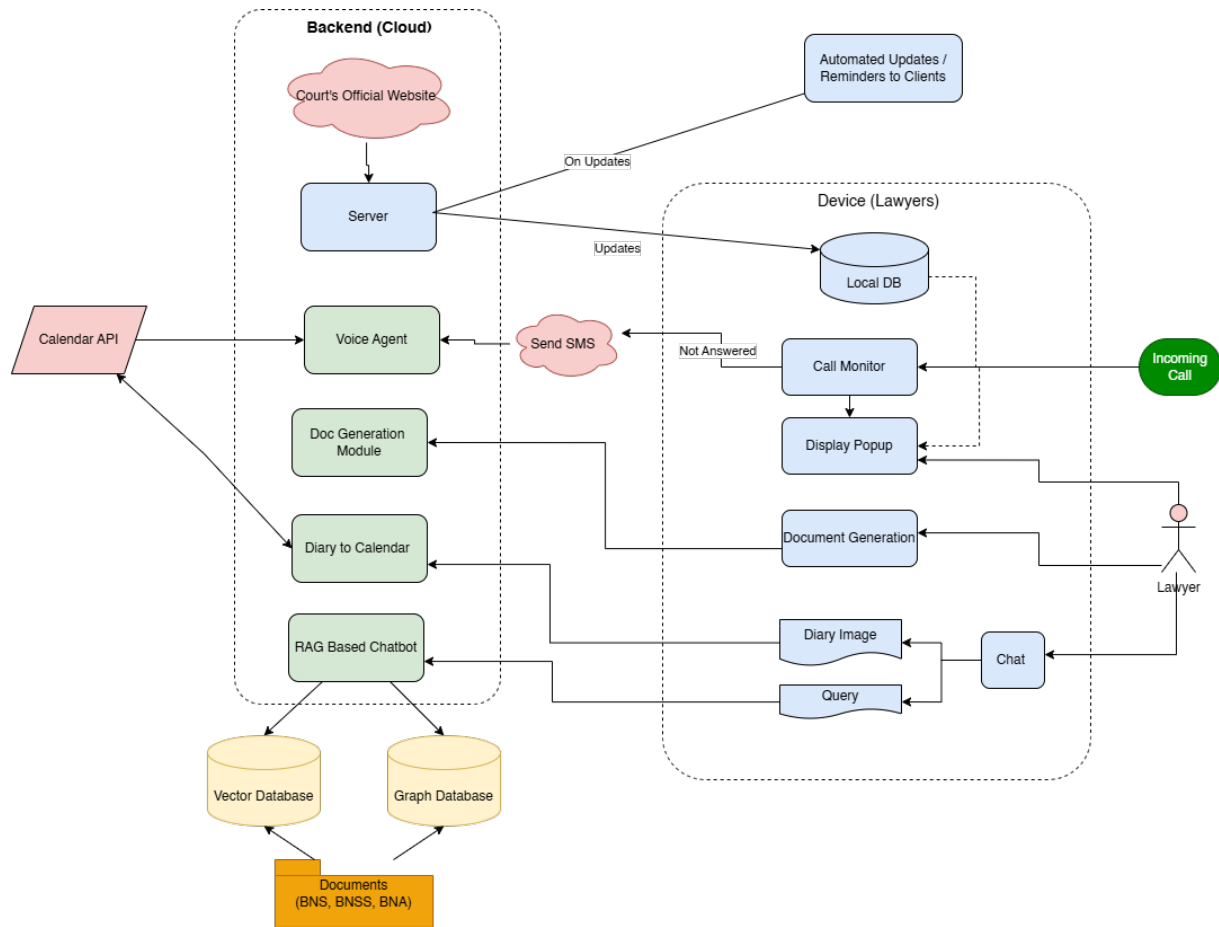


Figure 3.1: Full System Architecture Design

3.2 Component Design

3.2.1 Client Communication Module

The client communication module provides instant access to client case details during an incoming call by relying on a proactively updated local database. A backend Python service scrapes official court websites for updates, storing data in a cloud-based MongoDB database. When updates are detected, critical information, the latest case history, next hearing date, and CNR number is pushed to the Android app’s local Room database via Firebase Cloud Messaging (FCM) [11]. If a call is missed, the system sends a unique SMS link to a dedicated chatbot scoped strictly to that client’s case data. The chatbot also checks the lawyer’s Google Calendar, allowing the client to schedule a meeting automatically.

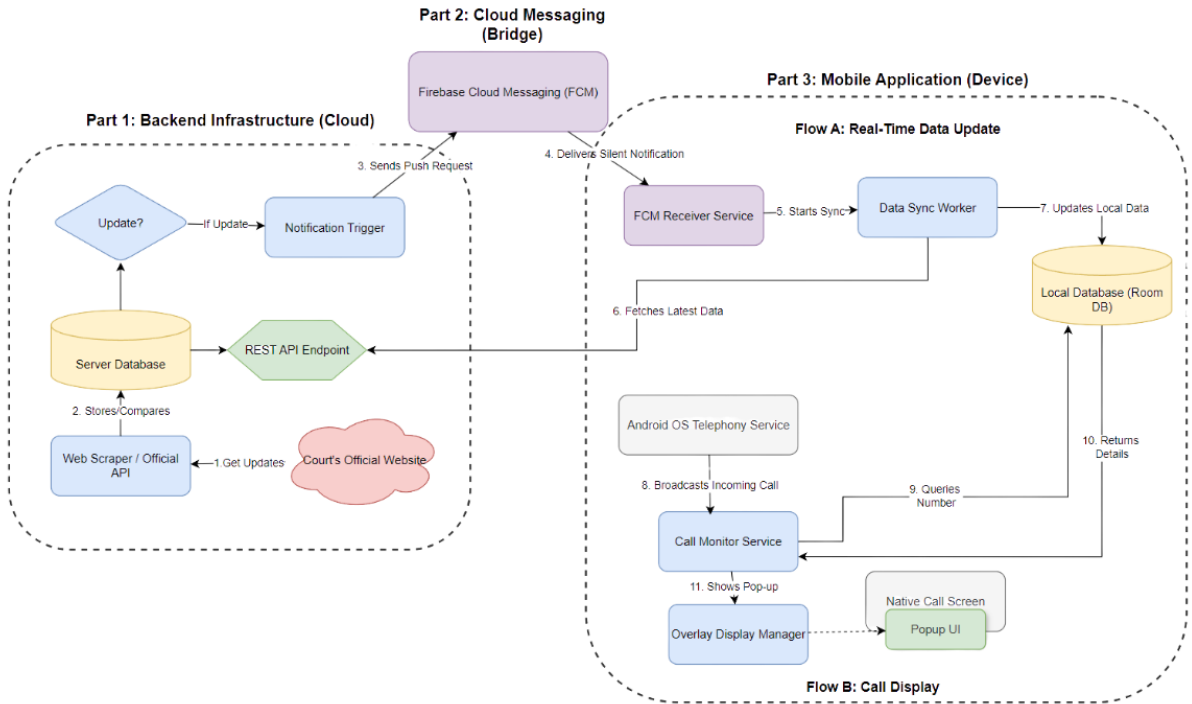


Figure 3.2: Component Design of Real-Time Context-Aware Call Popup System

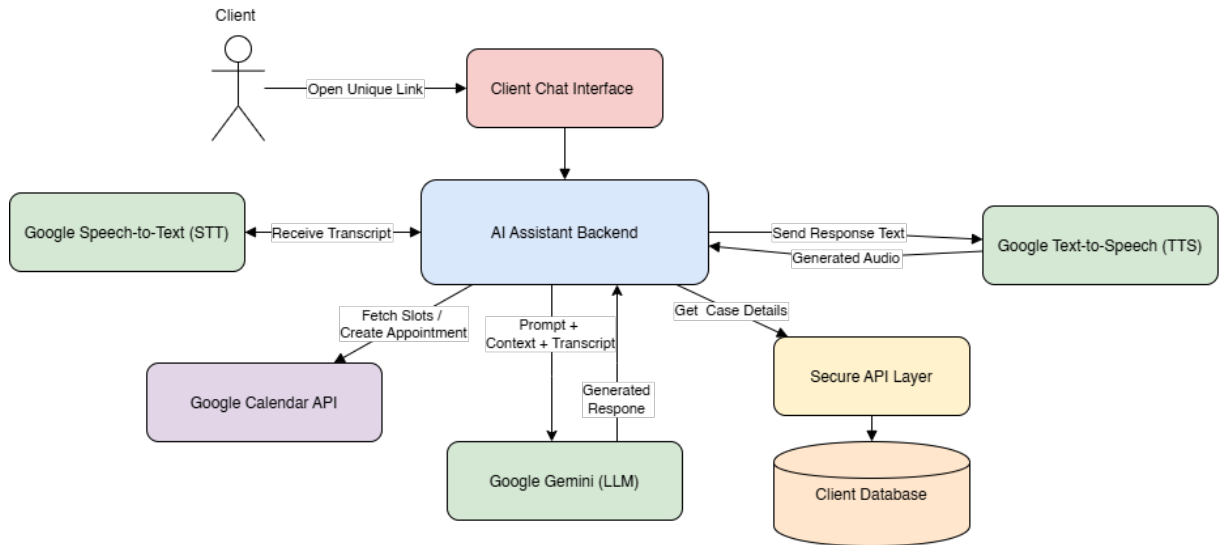


Figure 3.3: Component Design of Client Chat Interface

3.2.2 Document Generation Module

The Document Generation Module handles both individual and bulk document creation behind a template-driven engine. It is structured around four core components: the Template Repository, the Client Data Resolver, the AI Refinement Engine, and the

Document Rendering Engine.

The Template Repository stores legal templates as HTML files with placeholder fields. The Client Data Resolver queries MongoDB Atlas to fetch client records and injects details via global regular expression replacement. When a Word document of a legal document is imported, the AI Refinement Engine uses Google Gemini to identify and normalise variable fields into snake_case placeholders, enabling user to add templates. The rendering engine uses a Pell.js rich-text editor for formatting and exports the final document as PDF. For bulk tasks, the xlsx library parses an uploaded spreadsheet and generates one PDF per row.

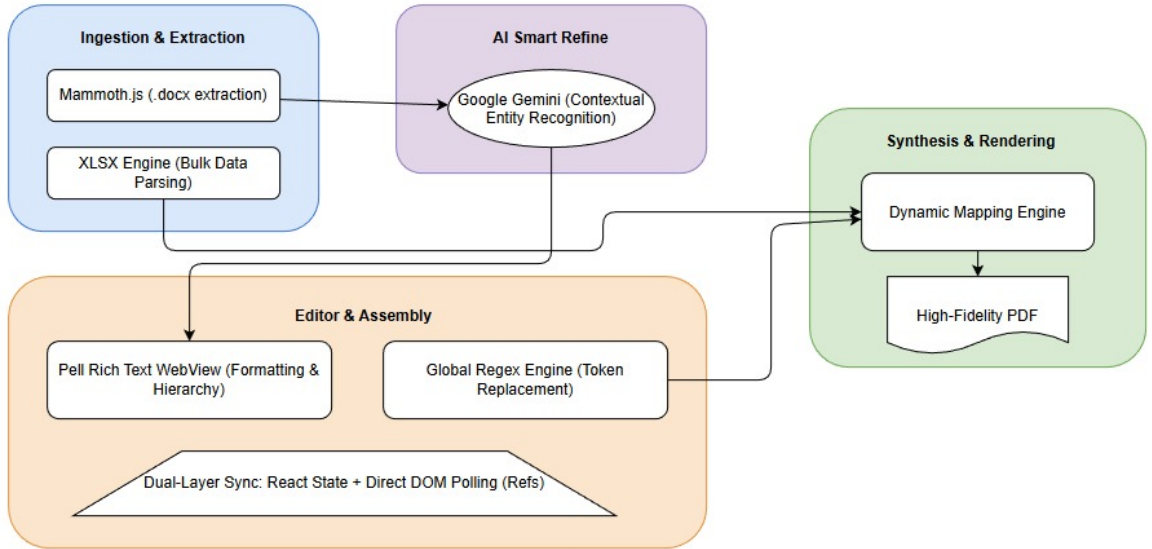


Figure 3.4: Component Design of Document Generation Module

3.2.3 RAG-Based Legal Chatbot Module

The RAG-Based Legal Chatbot Module is built on a hybrid GraphRAG architecture [1] combining a knowledge graph with dense vector search, grounded in the BNS [5], BNSS [6], and BSA [7]. The module has four components: the Document Ingestion Pipeline, the Knowledge Graph [10], the Vector Database, and the Query Processing Engine [9].

The Document Ingestion Pipeline performs section-level chunking, entity extraction, and relationship identification from the statutory texts. Legal entities such as offences,

punishments, and cross-references populate the Graph Database. The same chunked content is embedded using InLegalBERT [12] and stored in the Vector Database for similarity-based retrieval.

When a query is submitted, the Query Processing Engine runs a vector similarity search and traverses the Knowledge Graph to enrich retrieved content with related provisions. The combined context is passed to a large language model, generating a coherent, citation-aware response [1].

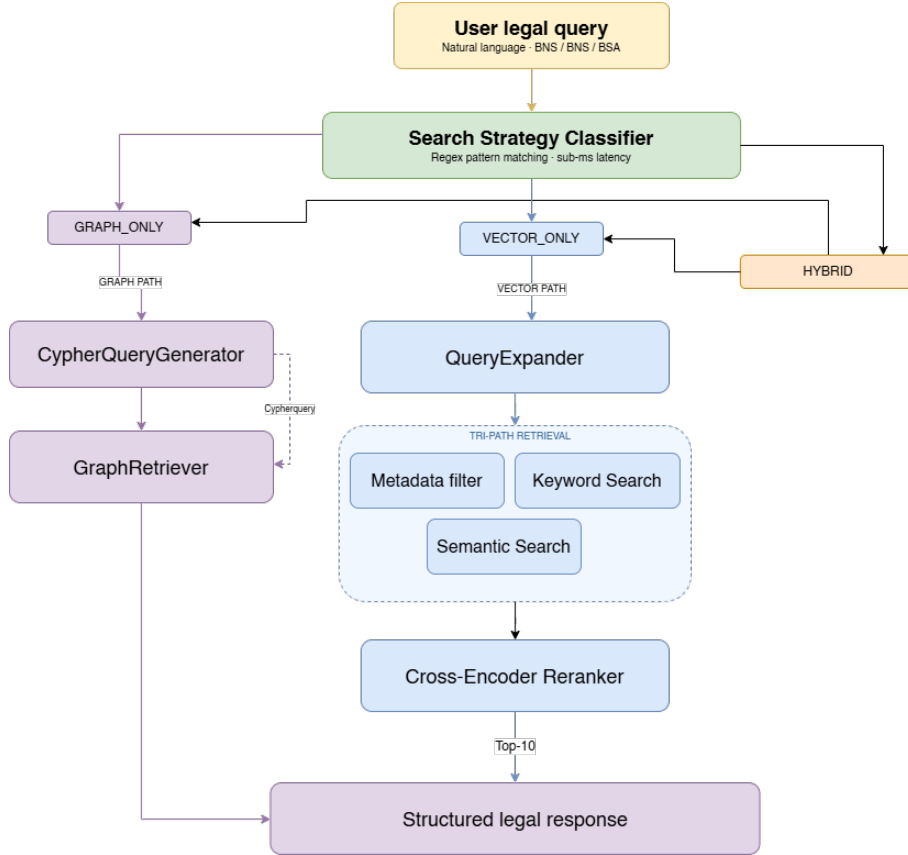


Figure 3.5: Component Design of GraphRAG based Chatbot

3.2.4 Diary to Calendar Module

The Diary to Calendar Module bridges paper-based hearing records and the automated calendar system. It has four sequential components: the OCR and Extraction Engine, the Court Data Scraper, the Reconciliation Engine, and the Calendar Integration Layer.

The OCR and Extraction Engine processes scanned diary images in Malayalam, English, or mixed script, extracting the client name, case number, court name, and hearing date. The Court Data Scraper retrieves official hearing schedules from the eCourts portal [8]. The Reconciliation Engine compares diary records against scraped data, applying a confidence scoring mechanism to resolve discrepancies and flag uncertain entries for manual review.

Verified records are passed to the Calendar Integration Layer, which maps each hearing onto the Calendar API and configures automated reminders. The local device database is updated simultaneously to keep all components in sync.

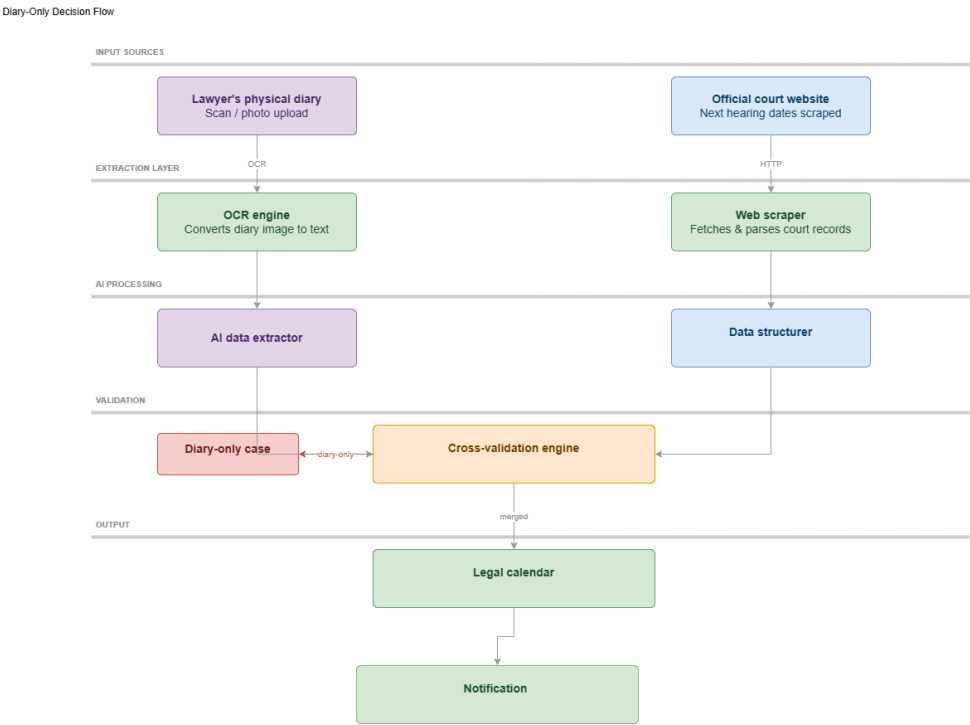


Figure 3.6: Component Design of Diary to Calendar

3.3 Tools and Technologies

Software Requirements:

- **Large Language Models (LLMs):** Gemini.
- **Mobile Application Framework:**
 - **Language:** Kotlin, React Native.
 - **UI Toolkit:** Jetpack Compose.
- **Databases:**
 - **Cloud Database:** MongoDB.
 - **Local Mobile:** Room DB.
 - **Vector:** Qdrant [13].
 - **Graph:** Neo4j [14].
- **Cloud & Backend Services:**
 - **Server Hosting:** AWS EC2, Render.
 - **Push Notifications:** Firebase Cloud Messaging (FCM) [11].
 - **Calendar Integration:** Google Calendar API.
- **AI Services (APIs):**
 - **Speech-to-Text (ASR):** Sarvam.
 - **Text-to-Speech (TTS):** Sarvam.
 - **Embedding Model:** InLegalBERT [12].

Hardware Requirements:

- **User Device:** Modern Android smartphone with internet connectivity.
- **Backend Infrastructure:** Cloud servers on Render, AWS, or Google Cloud for backend services, databases, and AI inference.

3.4 Work Breakdown and Responsibilities

- **Alan J Arackal:** Voice Agent & Pop-up during Calls.
- **Alsha Rose George:** Diary to Calendar & Knowledge Graph Construction.
- **Ahmed Abdul Majeed:** Document Generation.
- **Arya Vinod:** Legal Chatbot.
- **All Members:** Integration and Testing.

3.5 Key Deliverables

- **Automated Data Sync Engine:** A Python-based backend that scrapes court websites, stores data in MongoDB, and pushes hearing dates, CNR numbers, and case history to the mobile app via FCM [11].
- **Zero-Latency Caller ID:** An offline-first Android module using a continuously updated local Room database to instantly display client case details during incoming calls.
- **Automated Client Chatbot & Scheduler:** A missed-call handler that sends an SMS link to a case-specific chatbot, which answers client queries and schedules appointments into the lawyer’s Google Calendar.
- **GraphRAG Legal Chatbot:** A hybrid retrieval chatbot grounded in BNS, BNSS, and BSA using InLegalBERT [12] vector search over Qdrant [13] and knowledge graph traversal over Neo4j [14].
- **Bulk Document Generator:** A template-driven engine that auto-fills legal documents from client records and generates batches of PDFs from Excel input.
- **Diary to Calendar Pipeline:** An OCR-based module that extracts hearing data from scanned diary pages, reconciles it with eCourts data [8], and syncs verified schedules to Google Calendar with automated reminders.

3.6 Project Timeline

Table 3.1: Project Timeline

Phase	Activity	Duration
Phase 1	Requirements and Literature Survey	Month 1
Phase 2	System Design and Architecture	Month 2
Phase 3	Module Implementation	Month 3–4
Phase 4	Integration and Testing	Month 5
Phase 5	Evaluation and Report Writing	Month 6

3.7 Summary of Chapter

This chapter presented the system architecture and component design of NyayaSahayi. The four modules client communication, document generation, RAG-based legal chatbot, and diary-to-calendar conversion are built on a shared MongoDB backend with Firebase-based synchronisation [11] and eCourts integration [8]. The hybrid GraphRAG approach [1] combining InLegalBERT [12], Qdrant [13], and Neo4j [14] forms the intellectual core of the platform. The following chapter details the concrete implementation of each module.

Chapter 4

Results and Discussion

4.1 Introduction

This chapter presents the results obtained from the implementation and testing of *NyayaSahayi*. The system was evaluated across its four core modules: the Hybrid GraphRAG Legal Research Chatbot, the Real-Time Call Popup Module, the Intelligent Document Generation Module, and the Automated Docketing and Calendar Synchronisation Module. The findings confirm that the unified architecture successfully addresses the workflow fragmentation problem that motivates the project, delivering reliable performance across legal research, client communication, document automation, and schedule management.

4.2 GraphRAG Legal Research Chatbot

The proposed system, *NyayaSahayi*, was evaluated against distinct categories of legal queries spanning the BNS, BNSS, and BSA statutes. To optimize retrieval efficiency and accuracy, an intelligent query routing mechanism was developed to dynamically categorize incoming user queries into one of three execution paths based on their syntactic and semantic complexity:

- **Graph-Centric Retrieval:** Direct section-reference queries (e.g., “*What is Section 106 of the BNS?*”) were resolved through deterministic filtering and direct Knowledge Graph lookups. This approach bypassed unnecessary semantic processing, returning exact statutory provisions with minimal latency.
- **Vector-Centric Retrieval:** Situational and conceptual queries (e.g., “*What constitutes a sudden fight in the heat of passion?*”) were handled by a dense vector search over the Qdrant database using the *InLegalBERT* embedding model. This

semantic retrieval successfully surfaced relevant legal doctrines and exceptions that rigid keyword-based systems typically overlook.

- **Hybrid Search Strategy:** For multifaceted, multi-hop queries requiring both abstract legal reasoning and strict procedural facts (e.g., *“Five people commit robbery and kill a man. What is the punishment, and which court has jurisdiction?”*), the system executed parallel retrieval. The semantic vector space provided the nuanced statutory definitions, while a Text-to-Cypher pipeline concurrently traversed the Neo4j knowledge graph—mapping structured relationships across offence, court, punishment, and bail nodes.

To ensure optimal context quality, the candidates retrieved from the parallel paths were deduplicated and evaluated by a Cross-Encoder model. This reranking phase assigned confidence scores to prioritize the most contextually pertinent legal sections before they were passed to the generation phase.

A critical result of this hybrid architecture is the effective mitigation of Large Language Model (LLM) hallucinations. By strictly constraining the generative model to synthesise its final response solely from the fused context window—comprising both structured graph facts and verified statutory text—the system’s reliance on internal parametric memory was neutralized. Consequently, every generated output included explicit, highly accurate statutory citations securely grounded in the retrieved legislative data.

4.3 Real-Time Call Popup Module

The call popup module performed as designed during prototype testing. The mobile application successfully intercepted incoming calls, queried the local database, and rendered the case-context overlay consistently within 200 milliseconds well within the ringing window. The popup displayed the client name, next hearing date, case status.

The missed call workflow also operated correctly. When a call from a known client went unanswered, the backend generated a unique URL and dispatched an automated SMS. The client-facing chatbot, accessible via that link, correctly scoped all retrieval to the specific client’s case data, with no cross-client data leakage observed.

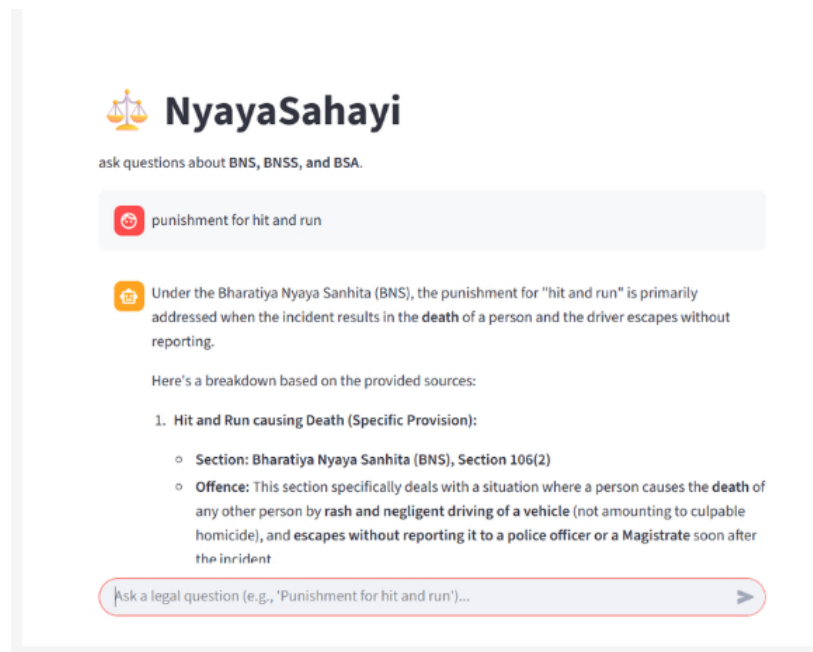


Figure 4.1: GraphRAG based Chatbot for Lawyers

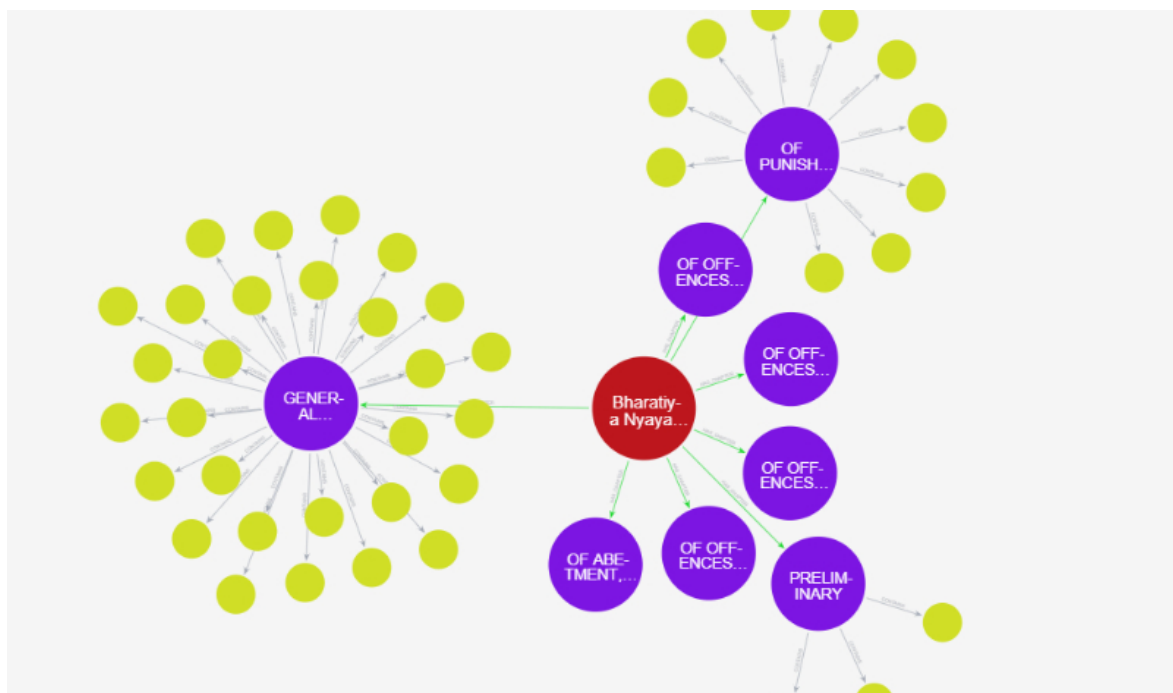


Figure 4.2: Knowledge Graph

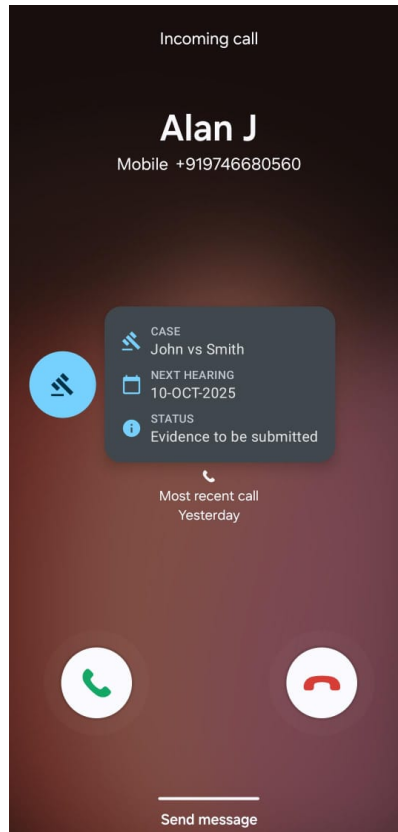


Figure 4.3: Pop Up During an Incoming Call

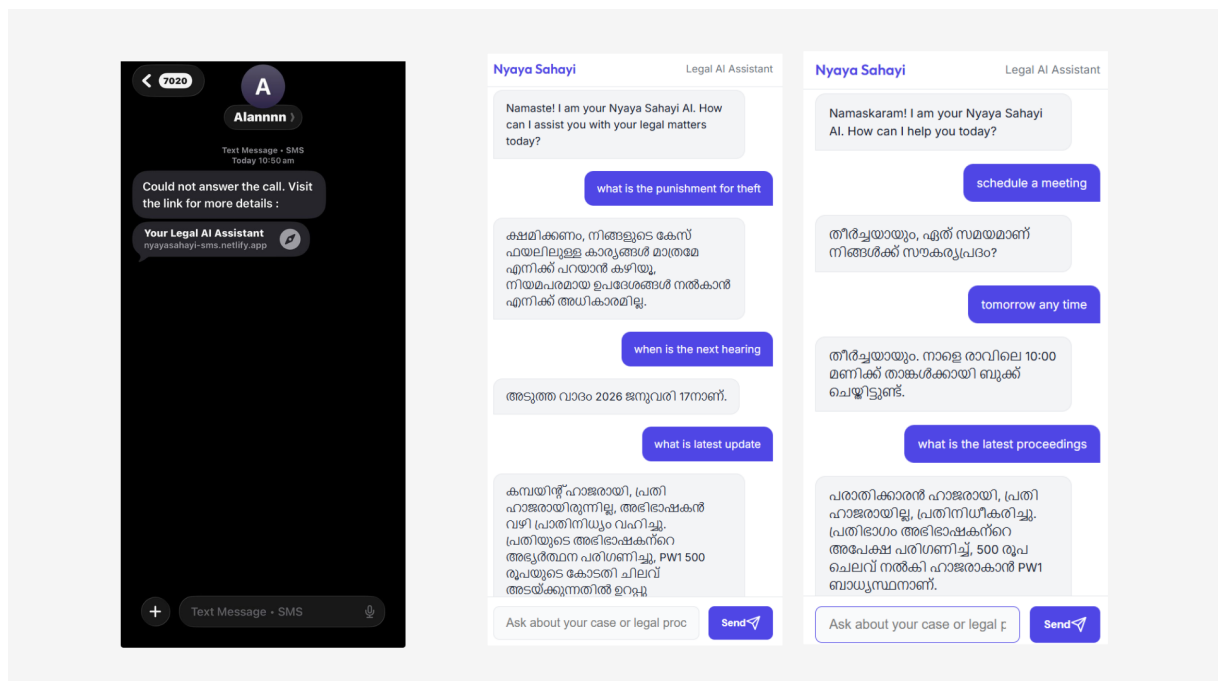


Figure 4.4: SMS with Unique URL and Client Chatbot Interface

4.4 Document Generation and Automation Module

The rich-text editor provided stable, mobile-responsive document drafting with professional formatting. The dual-check persistence mechanism (event-driven state and ref-based DOM polling) resolved asynchronous save issues, and the 10-second auto-save prevented content loss during testing. The bulk generation pipeline successfully ingested Excel data, mapped row-level values into HTML legal boilerplates, and exported correctly formatted PDF documents, significantly reducing the manual effort required for high-volume drafting tasks.

The template bootstrapping pipeline correctly identified both explicit placeholder markers via regex and implicit variable fields via Gemini AI extraction. The prompt constraints enforcing **snake_case** naming, preserving legal terminology, and stripping LLM artefacts produced clean, directly usable template structures.

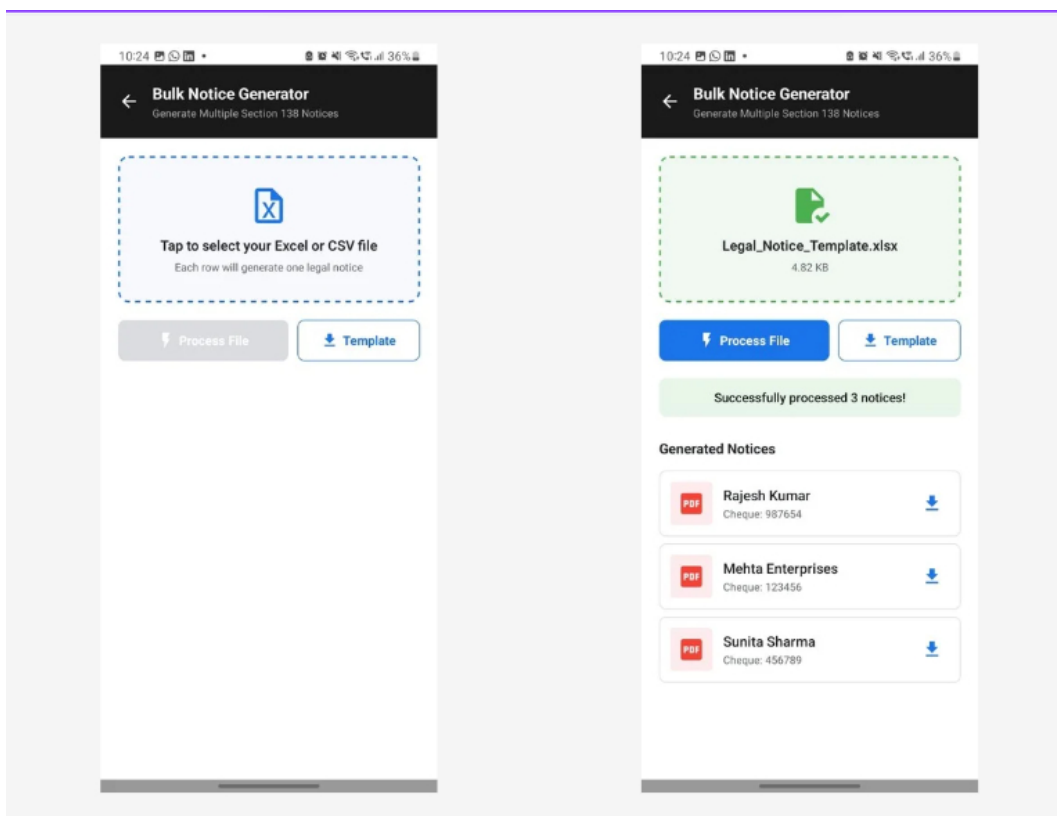


Figure 4.5: Bulk Document Generation

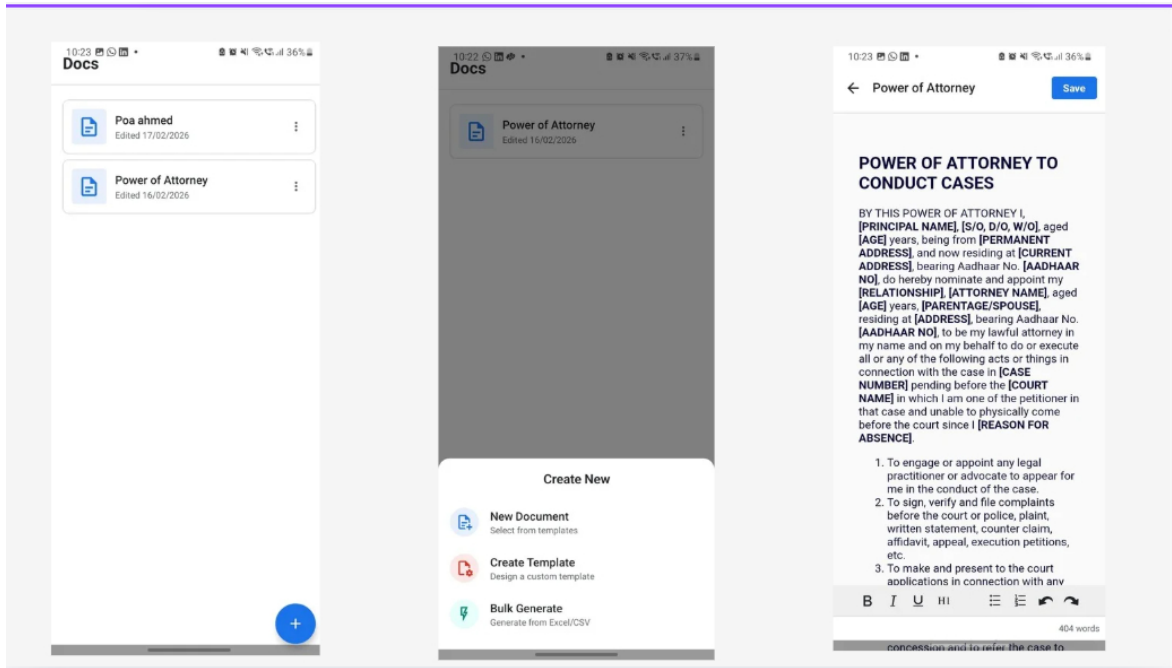


Figure 4.6: Generating Legal Documents from Templates

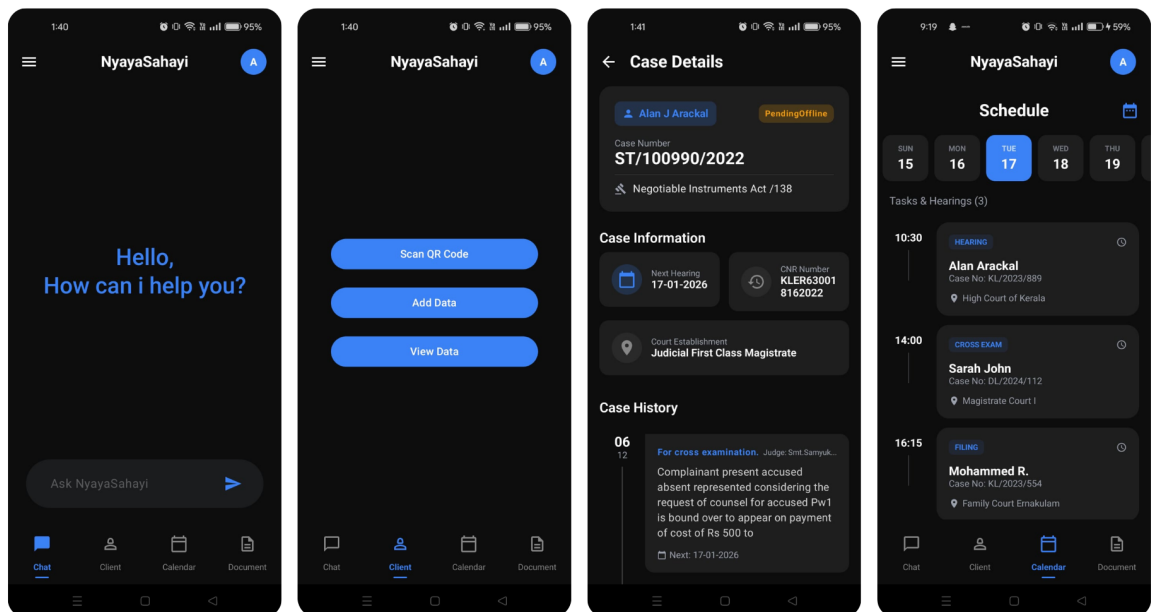


Figure 4.7: Mobile UI

4.5 Automated Docketing and Calendar Synchronisation

The Google Calendar integration successfully created and updated hearing-date events with full case metadata (case name, number, court, and stage). Updates to hearing dates sourced from the e-Courts crawler were reflected in the calendar automatically, maintaining consistency between the docketing records and the lawyer’s schedule. Configurable advance reminders were created for each hearing, providing a reliable safeguard against missed dates.

4.6 Discussion

4.6.1 Strengths of the Unified Architecture

The most significant outcome of this work is the demonstration that integrating these four modules into a single platform yields benefits that no standalone tool can replicate. Research findings from the chatbot can be cited directly in a document being drafted in the editor. The case context shown in the call popup is sourced from the same data store that feeds the calendar, ensuring consistency. The client-facing chatbot operates over live case data without any additional entry by the lawyer. These cross-module interactions represent the core value proposition of *Nyayasahayi*.

4.6.2 Limitations

The following limitations were identified during the evaluation:

- **Corpus Coverage:** The knowledge graph and vector database currently cover only the three principal Indian criminal codes. Civil codes, state amendments, and judicial precedents are not yet included, limiting the depth of research support.
- **Text-to-Cypher Robustness:** Highly ambiguous natural language queries occasionally produced syntactically incorrect Cypher queries, requiring fallback to vector-only retrieval. Fine-tuning on a legal-domain Cypher dataset is a clear next step.
- **API Dependency:** The call popup and docketing modules depend on e-Courts API availability. Courts without programmatic access require manual data entry,

reducing automation benefits.

- **Platform Coverage:** The current implementation targets Android. Full iOS feature parity particularly for telephony interception and system overlays requires significant additional platform-specific work.

4.7 Summary

The results validate the core objective of *Nyayasahayi*: a unified AI-driven platform can effectively centralise legal research, client communication, document automation, and schedule management into a single coherent workflow. The GraphRAG chatbot delivers citation-grounded statutory responses; the call popup provides sub-200ms contextual assistance; the document module automates both individual and bulk drafting; and the docketing module ensures reliable calendar synchronisation. The limitations identified — corpus breadth, query translation robustness, API dependency, and platform coverage — provide a concrete roadmap for future development.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

This project presented *Nyayasahayi*, a unified AI-driven legal assistance platform designed to eliminate the workflow fragmentation that burdens modern legal practitioners. By integrating four core modules a Hybrid GraphRAG Legal Research Chatbot, a Real-Time Call Popup, an Intelligent Document Generation Module, and an Automated Docketing and Calendar Synchronisation Module the system centralises legal research, client communication, document drafting, and schedule management into a single cohesive platform tailored to the Indian legal context.

The GraphRAG architecture, combining InLegalBERT-powered vector retrieval over Qdrant with structured knowledge graph traversal over Neo4j, effectively mitigated LLM hallucination by grounding every response in verified statutory text from the BNS, BNSS, and BSA. The call popup module delivered context-aware case summaries within 200 milliseconds. The document module automated both individual drafting and bulk document generation. The docketing module ensured reliable hearing-date synchronisation with Google Calendar.

The results validate that a purpose-built, domain-specific AI system delivers more reliable and practical value in a high-stakes professional domain than adapting general-purpose AI tools. The unified data layer shared across all modules creates workflow continuities research flowing into drafts, call context consistent with the calendar that no collection of standalone tools can replicate.

5.2 Future Scope

Several directions are identified for future development:

- **Expanded Legal Corpus:** Extending the knowledge base to cover civil legislation, special statutes, Supreme Court and High Court judgments, and state-specific amendments to broaden research support beyond criminal law.
- **Improved Text-to-Cypher Translation:** Fine-tuning the query translation layer on a legal-domain dataset to reduce fallback frequency on ambiguous queries.
- **Multi-Turn Conversational Memory:** Adding conversation state to allow the chatbot to handle follow-up queries without requiring the user to restate context.
- **iOS Support:** Developing a fully featured iOS counterpart using CallKit to extend the platform beyond Android.
- **Multilingual Support:** Enabling the chatbot and document module to operate in Hindi and major regional languages to improve accessibility across India.
- **Web Dashboard:** A browser-based interface for multi-lawyer firms supporting shared case management and collaborative document editing.
- **Deeper e-Courts Integration:** Tighter coupling with India’s National Judicial Data Grid (NJDG) for automatic order ingestion and direct e-filing support.

As the system matures toward broader deployment, compliance with India’s Digital Personal Data Protection Act (DPDPA) 2023 and a formal security audit of the client data partitioning architecture will be essential priorities.

References

- [1] P. Wang *et al.*, “Hypa-rag: A hybrid parameter adaptive retrieval-augmented generation system for ai legal and policy applications,” *arXiv preprint arXiv:2409.09046*, 2024.
- [2] G. Lai, F. Riva, and M. Palmirani, “An llm-assisted etl pipeline to build a high-quality knowledge graph of the italian legislation,” in *Proc. International Conference on Legal Knowledge and Information Systems (JURIX)*, 2024.
- [3] K. Hamdan *et al.*, “Toward low-latency end-to-end voice agents for telecommunications using streaming asr, quantized llms, and real-time tts,” *arXiv preprint arXiv:2501.02983*, 2025.
- [4] S. Shalini R and J. Vishwakarma, “Legal document generation: Harnessing large language models for diverse legal workflows,” *Indian Journal of Law and Legal Research*, vol. VI, no. IV, pp. 187–207, 2024.
- [5] Government of India, “Bharatiya nyaya sanhita,” Ministry of Law and Justice, New Delhi, 2023, [Online]. Available: <https://legislative.gov.in>.
- [6] —, “Bharatiya nagarik suraksha sanhita,” Ministry of Law and Justice, New Delhi, 2023, [Online]. Available: <https://legislative.gov.in>.
- [7] —, “Bharatiya sakshya adhiniyam,” Ministry of Law and Justice, New Delhi, 2023, [Online]. Available: <https://legislative.gov.in>.
- [8] eCourts Mission Mode Project, “National judicial data grid,” Department of Justice, Government of India, [Online]. Available: <https://njdg.ecourts.gov.in>.
- [9] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

- [10] Y. Pan *et al.*, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [11] Google LLC, “Firebase cloud messaging documentation,” 2024, [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>.
- [12] E. J. Choi *et al.*, “InLegalBERT: A pre-trained language model for indian legal text,” in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2022.
- [13] Qdrant Team, “Qdrant vector database documentation,” 2024, [Online]. Available: <https://qdrant.tech/documentation>.
- [14] Neo4j Inc., “Neo4j graph database documentation,” 2024, [Online]. Available: <https://neo4j.com/docs>.

Appendix A: Presentation



100% Evaluation Presentation

NyayaSahayi

Guide: Ms. Janani K

PRESENTED BY:

AHMED ABDUL MAJEED (U2203019)

ALAN J ARACKAL (U2203020)

ALSHA ROSE GEORGE (U2203032)

ARYA VINOD (U2203053)

Contents

01	Problem Definition
02	Project Objective
03	Purpose and Need
04	Literature Survey
05	Proposed System
06	Module Description
07	Results
08	Conclusion



Introduction

The Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS), and Bharatiya Sakshya Adhinyam (BSA) were introduced in 2023 to replace the IPC, CrPC, and Indian Evidence Act, aiming to modernize and simplify criminal justice.

Problem Definition

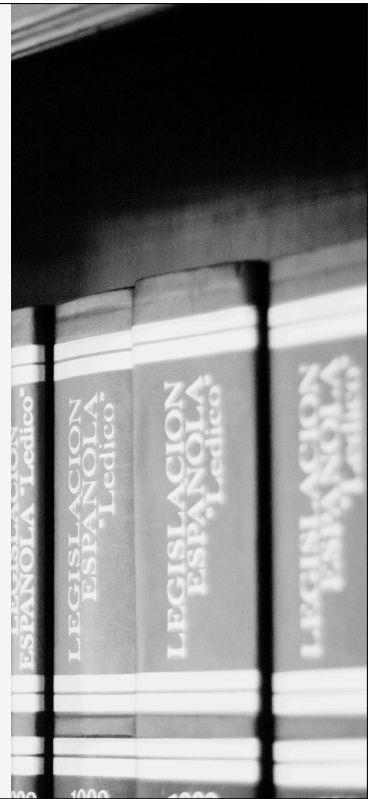
Legal practitioners and law firms often face **inefficiencies in managing appointments, responding to client queries, and maintaining up-to-date schedules.**

Manual management of physical or digital diary entries for hearings and meetings is time-consuming and prone to errors. **Drafting repetitive legal documents** further adds to the workload.



Objectives

- Develop an **RAG based Assistant** to answer user queries.
- Enable quick generation of **repetitive legal documents** from predefined templates.
- Provide responses to clients with relevant case information with the help of a **voice agent**.
- Send clients **notifications/alerts** with updated case status.
- Display **case details as a popup** when the client calls.
- Scan QR code to get case details.
- **Automate the extraction of hearing dates** from diary entries or documents and update the calendar.



Purpose and Need

Purpose

The purpose of Nyayasahayi is to create a **unified digital ecosystem** that streamlines the complete workflow of legal practitioners. It aims to **centralize legal research, client interaction, document drafting, and case scheduling** into one intelligent platform, ensuring accuracy, efficiency, and reliability in daily legal practice.

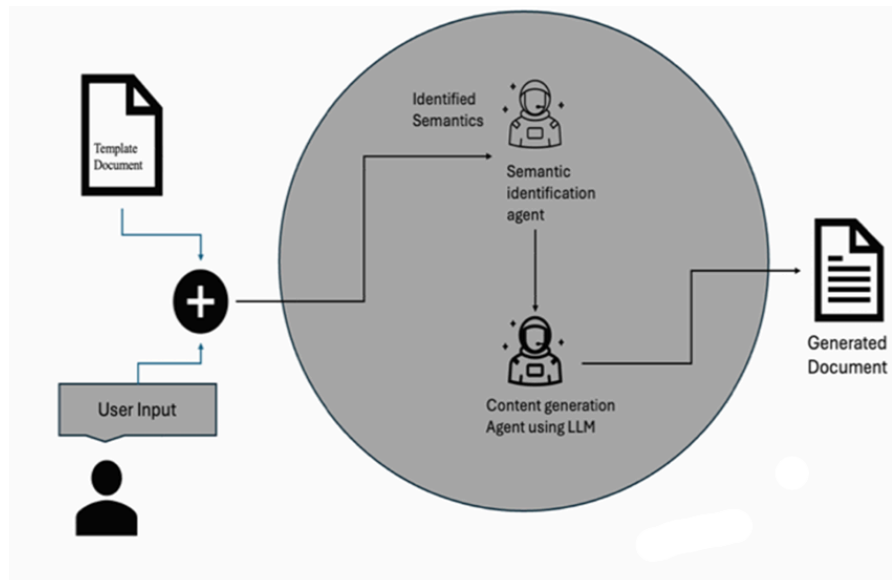
Need

Legal professionals currently rely on **multiple disconnected tools**, leading to time loss, cognitive overload, and administrative inefficiencies. Additionally, **general AI tools risk hallucinations and lack domain grounding**, which is critical in law. There is a strong need for a reliable, integrated system that reduces manual work, minimizes errors, and improves productivity while maintaining factual correctness in legal research and documentation.

Literature Survey

HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications

- It aims to address limitations of Large Language Models (LLMs) such as outdated knowledge, hallucinations, and poor reasoning in complex legal contexts, as well as challenges faced by traditional RAG systems like retrieval errors and ineffective context integration.
- Key components include a query complexity classifier for adaptive parameter tuning, a hybrid retrieval approach combining dense, sparse, and knowledge graph methods, and a comprehensive evaluation framework.
- The system was tested using NYC Local Law 144 as its test case.
- Its primary goal is to enhance retrieval accuracy, response fidelity, and contextual precision in high-stakes legal and policy applications.



An LLM-assisted ETL pipeline to build a high-quality knowledge graph of the Italian legislation

The paper proposes a structured LLM assisted ETL pipeline for constructing a high quality legislative Knowledge Graph from official sources.

Extract Phase:

- Legislative documents are obtained in structured XML format .This ensures reliable and standardized input data.

Transform Phase:

- XML elements such as laws, articles, paragraphs, citations, and amendments are systematically mapped into graph entities. Metadata is cleaned, normalized, and validated before graph construction.

Load Phase:

- The transformed data is loaded into a Property Graph database implemented using Neo4j. Schema constraints are applied to ensure structural consistency.

The system adopts a Property Graph model, where both nodes and relationships can store multiple properties.

Main Node Types:

- Law nodes representing legislative acts
- Article nodes modeled separately for fine-grained granularity
- Annex or attachment nodes when applicable

Relationship Types:

- CONTAINS (Law → Article)
- CITES (reference between legal elements)
- AMENDS (modification relationships)
- REPEALS (abrogation relationships)

Relationships also store properties such as temporal validity and reference context, enabling modeling of legal evolution over time.

BNS Mitra – RAG-Optimized LLM Based Legal Virtual Assistant

- Proposes an AI-powered legal assistant for interpreting provisions of Bharatiya Nyaya Sanhita (BNS).
- Integrates Large Language Model (LLM) with Retrieval-Augmented Generation (RAG) for accurate legal responses.
- Uses LLaMA 2 as the base model for natural language understanding.
- Implements FAISS for semantic vector search and efficient legal document retrieval.
- Achieves approximately 87% accuracy in mapping user queries to relevant BNS sections.
- Enhances accessibility to legal information while highlighting challenges in handling complex case narratives.

Proposed Method

- **GraphRAG-Based Legal Research Chatbot**

The system uses a Graph Retrieval-Augmented Generation (GraphRAG) architecture that grounds responses in verified statutes and case law. By structuring legal knowledge as a graph and retrieving relevant precedents before generating responses, it significantly reduces hallucinations and improves factual reliability—an essential requirement in legal practice.

- **Real-Time Context-Aware Call Popup System**

During client interactions, the system automatically retrieves relevant case details and displays them in real time. This ensures lawyers have contextual information readily available, improving response accuracy and professionalism.

- **Intelligent Document Editor**

The platform includes a smart document drafting module capable of auto-filling legal templates using structured case data. This reduces repetitive drafting work and minimizes manual errors.

- **Automated Docketing & Calendar Synchronization**

The system synchronizes court hearing dates and deadlines directly with Google Calendar, ensuring lawyers never miss critical case events.

Functional Requirements

Voice Agent & Client Calls

- Handle Incoming Calls: If unanswered, send SMS with a unique link
- Provide Information: Offer case updates and schedule appointments for clients.
- Caller ID Popup: Display a popup with relevant case details when a known client calls.

RAG-Based Legal Chatbot

- Answer Queries: Allow lawyers to ask legal questions in a chat interface.
- Retrieve & Cite: Fetch relevant sections from legal documents (BNS, BNSS, etc.)

Automated Document Generation

- Edit predefined templates to create new documents.
- Create repetitive documents like notices in bulk.
- Enable direct download of the finalized document.

Diary Extraction & Calendar Integration

- Capture & Scan: Allow users to upload or take photos of diary pages.
- Extract Details: Use OCR to automatically identify and extract text, dates, times, and event titles.
- Confirm & Save: Let users edit extracted information before saving it directly to their device calendar.

Modules

**Voice Agent
Phone Popup**

**RAG Based
ChatBot**

**Document
Generation**

**Calendar
and
Extraction
from diary**

Popup During Call

This system displays a **pop-up with a client's latest case details** the moment their call comes in. It uses a server to push **real-time updates** to your phone, ensuring the information is always current and the **pop-up appears instantly**.

- Database :
 - MongoDB (Cloud)
 - Room db (Local)
- Push Notifications: Firebase Cloud Messaging (FCM)
- Server Hosting: DigitalOcean, AWS EC2, or Heroku,Render

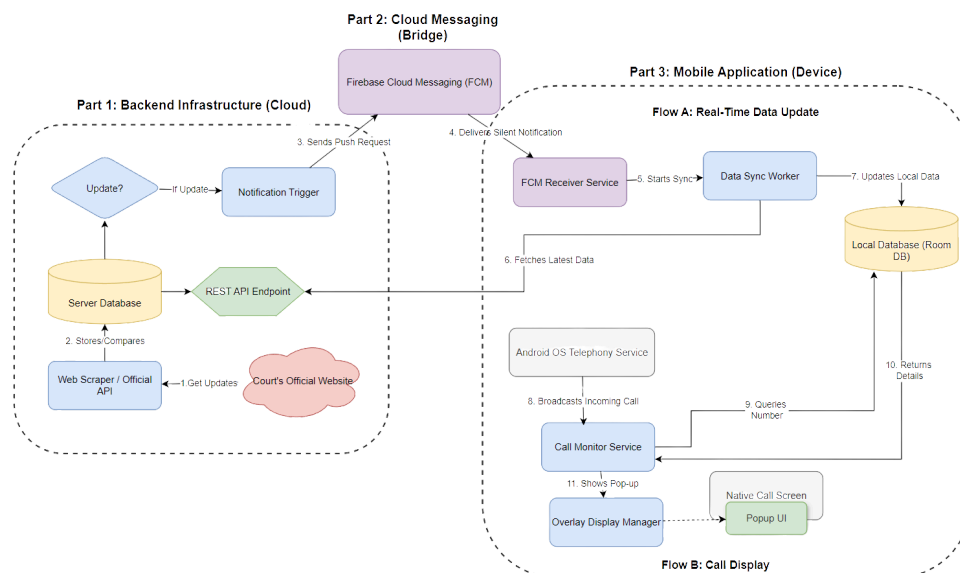


Fig 1: Architecture of Real-Time Context-Aware Call Popup System

Voice Agent

This system is an **AI agent** that automatically **answers client queries** in Malayalam, acting as a **virtual legal assistant** for your clients **24/7**. It **provides case details** by checking a database and **schedules new appointments** directly into your Google Calendar.

- ASR : OpenAI Whisper, Google Speech-to-Text, Indic Conformer, Sarvam, AudioX
- LLM : Gemini 2.5-flash /gemma
- TTS : Google Text-to-Speech, Eleven Labs, Sarvam
- Google Calender API

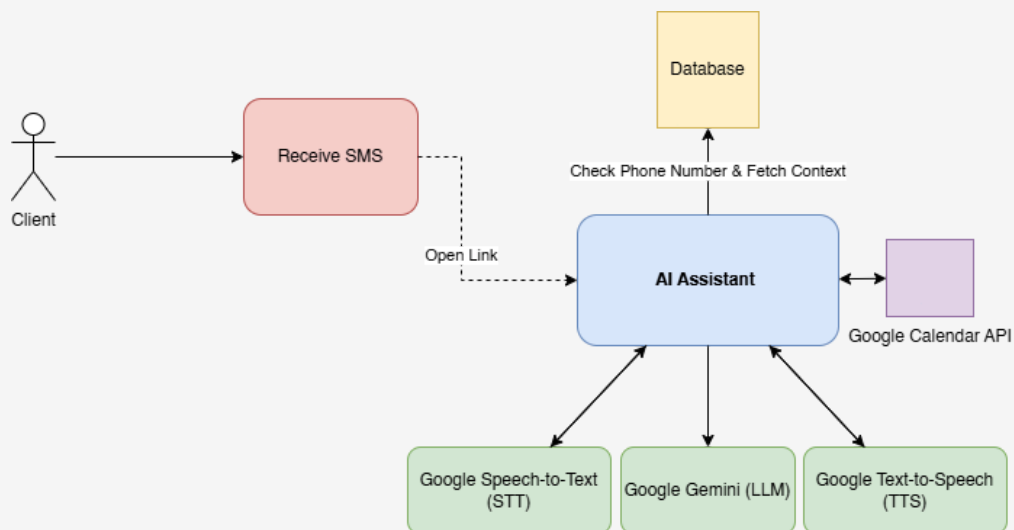
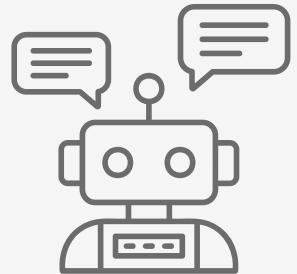


Fig 2: Architecture of Voice Agent

RAG Based ChatBot

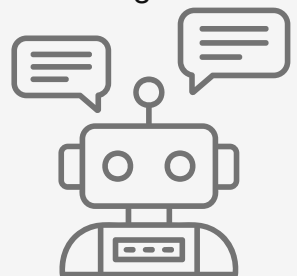
This system is an AI-powered legal assistant designed specifically for lawyers to answer legal queries accurately and contextually. It retrieves relevant information from **BNS, BNSS, and BSA**, processes it using Retrieval-Augmented Generation (RAG), and generates reliable responses with proper legal citations. The chatbot ensures faithfulness to Indian law and helps lawyers save time in legal research and case preparation.

- **Knowledge Source:** Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS), Bharatiya Sakshya Adhiniyam (BSA), and related judgments
- **Embedding Model:** law-ai/InLegalBERT
- **Vector Database:** Qdrant
- **Framework:** custom python RAG pipeline
- **LLM (Generator):** Gemini



RAG Based ChatBot

- **Data Collection**
Collect raw legal documents — BNS, BNSS, BSA (PDF)
- **Text Extraction**
Extract clean texts from PDF's using Docling
- **Data Cleaning & Preprocessing**
Remove headers, footers, page numbers, and normalize sections
- **Embedding Creation**
Convert chunks into numerical vector embeddings using InLegalBERT embeddings
- **Text Chunking**
Split text into smaller chunks (per section/sub-section)
- **Vector Database Storage**
Store embeddings for semantic retrieval in Qdrant



Knowledge Graph Construction

- Purpose: Leverage knowledge graphs to extract and connect information from project-related documents for better insights and decision-making.
- Data Source: PDF files of bns,bnss,bsa.
- Process:
 - PDF Extraction: Convert PDF content into structured text using OCR or PDF parsers.
 - Entity Recognition: Identify key entities like concepts, tasks, stakeholders, and resources.
 - Relationship Extraction: Detect relationships between entities to form meaningful connections.
 - Graph Construction: Build a knowledge graph linking entities and relationships for visual and query-based exploration.
 - Applications: Supports querying, reasoning.

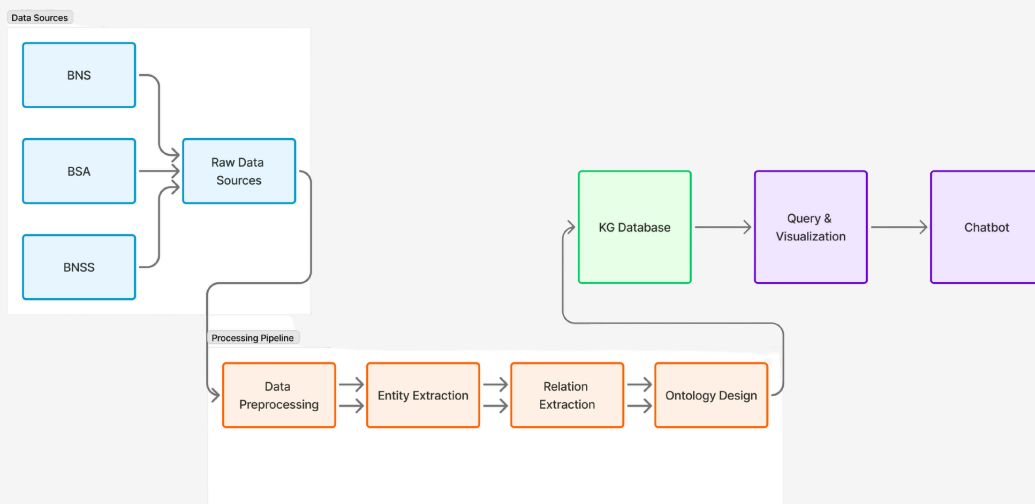


Fig 3 : Pipeline for knowledge graph construction

RAG Based ChatBot

STEPS

- **Query Input**
Lawyer enters legal question through the chat UI
- **Query Preprocessing**
Clean and tokenize the user query
- **Context Retrieval**
Retrieve top-k relevant chunks from vector DB
- **Answer Generation (RAG)**
Combine retrieved context with user query for response generation.
- **Backend Integration**
Process query, run RAG pipeline, and return result to UI

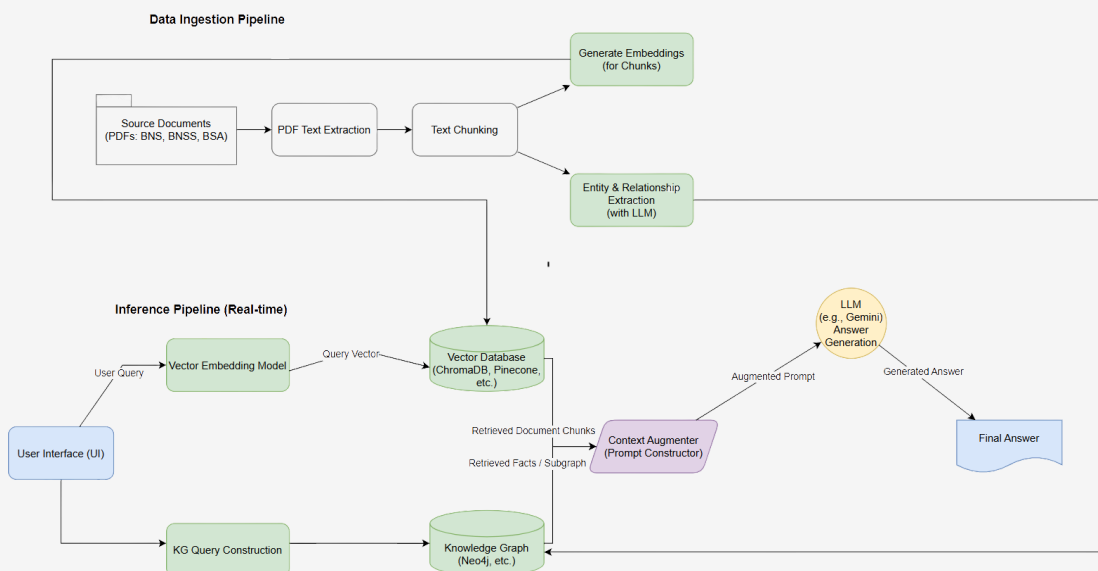
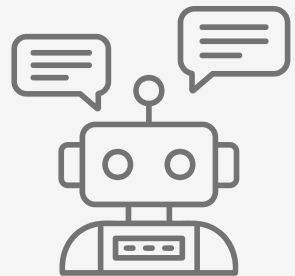


Fig 4 : Pipeline for RAG based Chatbot

Document Generation

This module is a comprehensive legal document generation system designed to simplify the creation, management, and bulk processing of documents.

- Database: MongoDB Atlas
- Frontend: React native, JavaScript, pell.js, mammoth, xlsx
- Backend: Node.js, Express.js
- External AI API: Gemini



Document Generation

Document Editor

- The system utilizes a customized react-native-pell-rich-editor WebView to provide the advanced formatting such as header hierarchies and bolding required for professional legal documentation.
- To ensure 100% data reliability, the editor combines real-time React state tracking with direct DOM polling via refs to bypass asynchronous state lag during critical actions.
- Automates document assembly by mapping client data to placeholders via a searchable modal and using global regex to instantly update the live editor with the new information.

Template creation

- The system uses mammoth.js to semantically extract content from .docx files .
- Powered by Google Gemini, this layer moves beyond simple pattern matching to identify legal variables.
- A specialized system prompt forces the AI to use consistent `{{snake_case}}` tags for variables.

Bulk generation of notices

- The system uses the xlsx engine to parse tabular data, combined with a validation layer .
- A dynamic engine injects row-level data into HTML legal boilerplates, enabling the high-speed generation of hundreds of notices.
- The mapped content is rendered into downloadable pdfs.

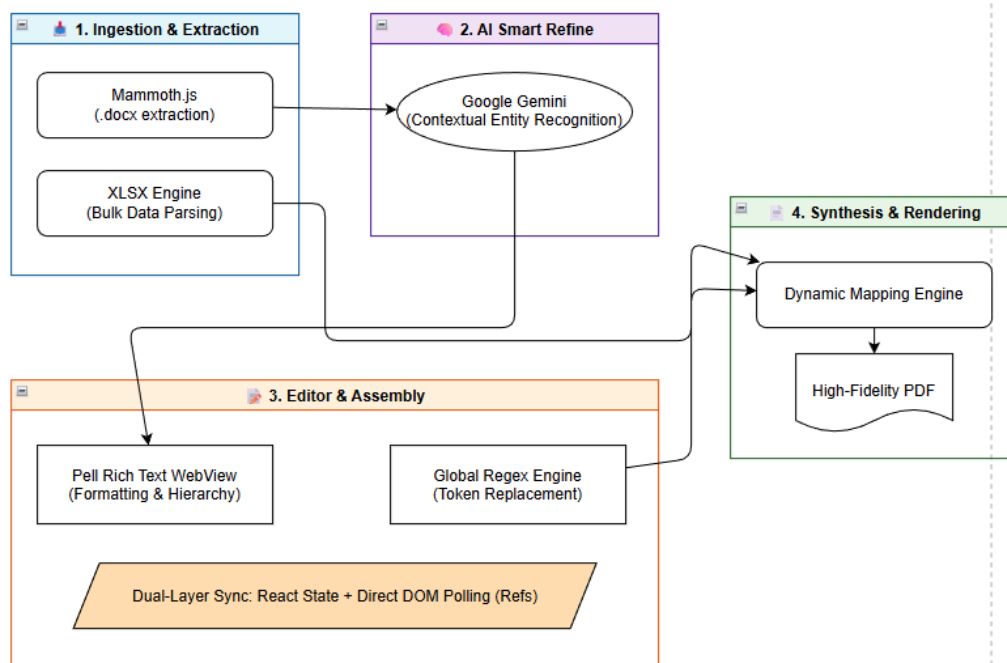


Fig 5: Architecture of Document Generation Module

Diary Image to Calender

This system helps you take a photo of your diary page and it automatically adds those events into calender

- Take a photo of your diary page.
- Extracts the dates and case details.
- Checks with the latest updates to see that none of the cases are missed.
- It creates a calendar event using that information.

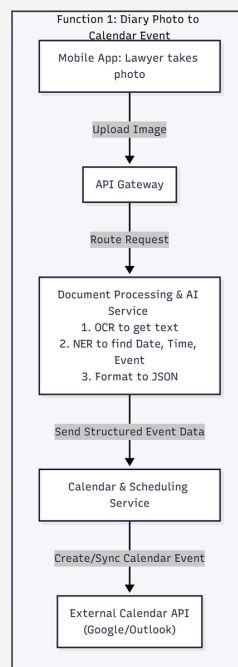
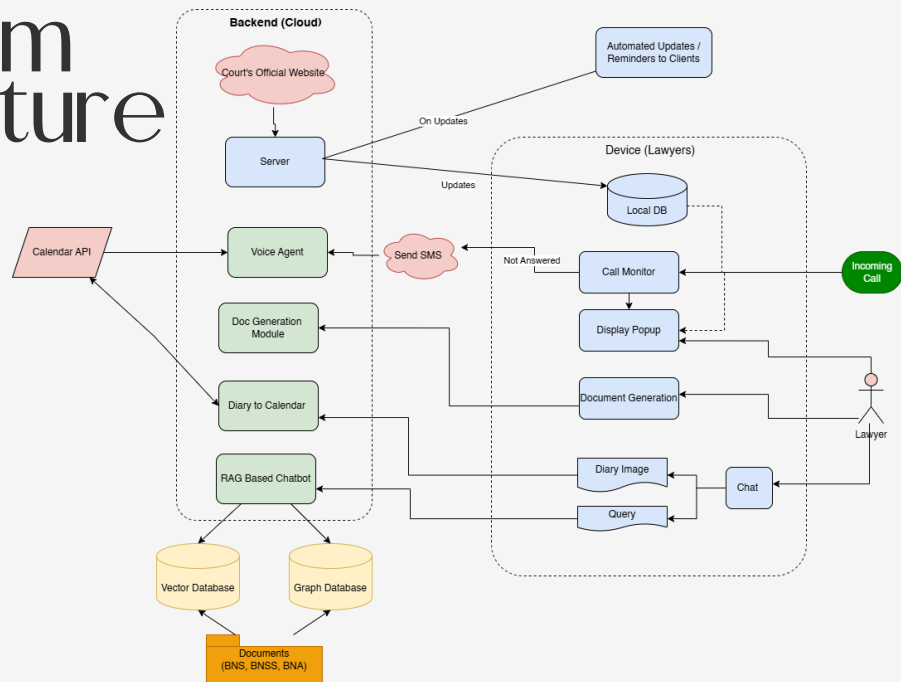
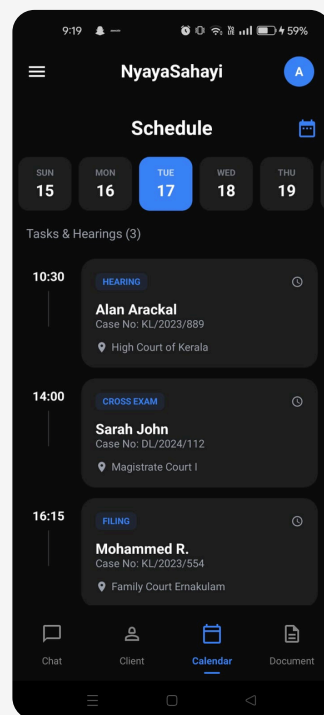
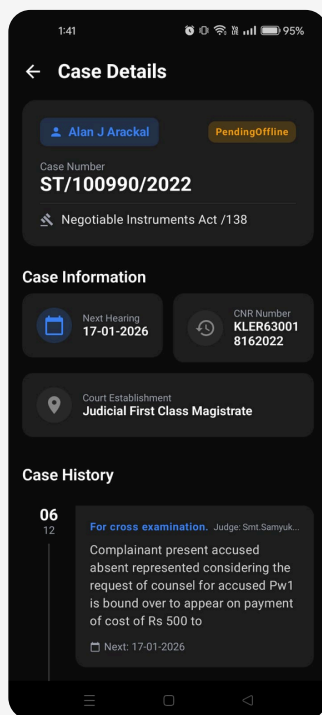
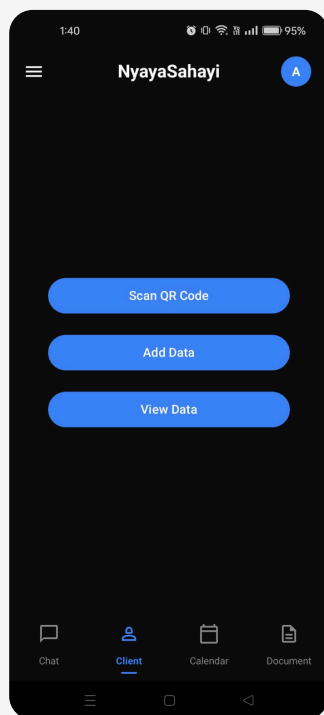
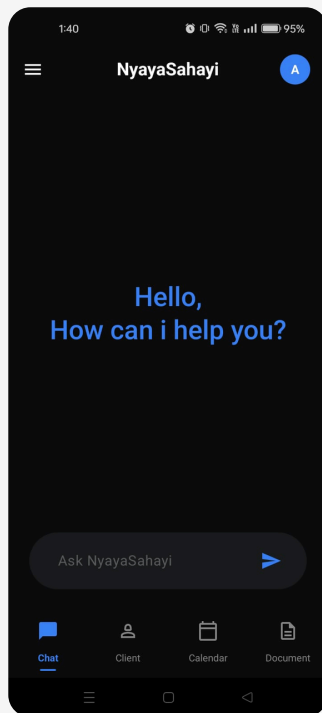


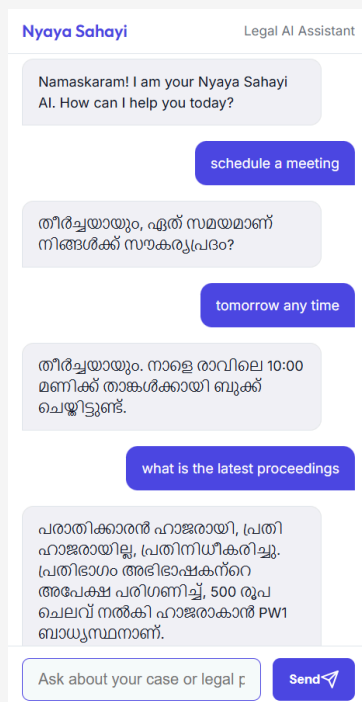
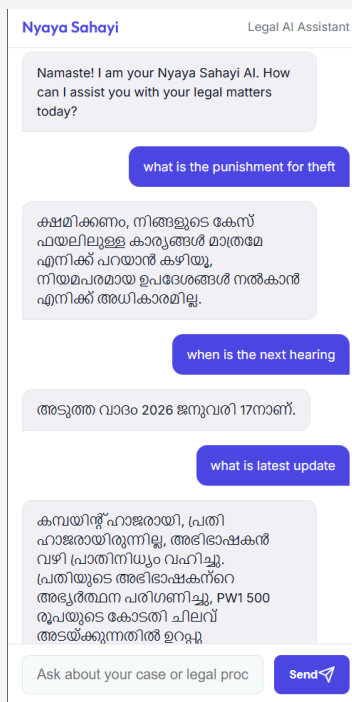
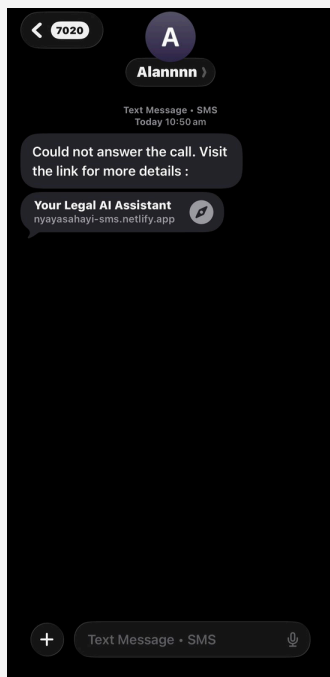
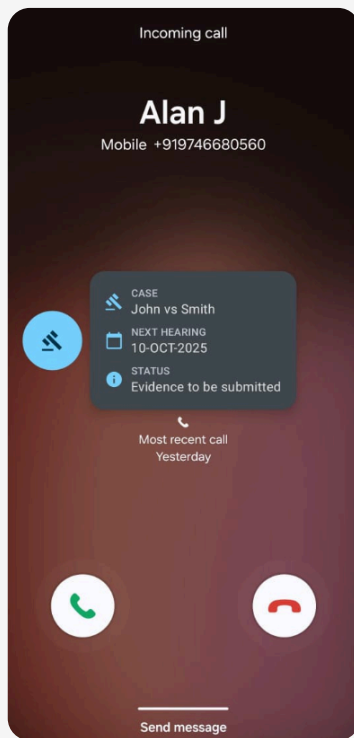
Fig 6: Architecture of Image to Calendar Module

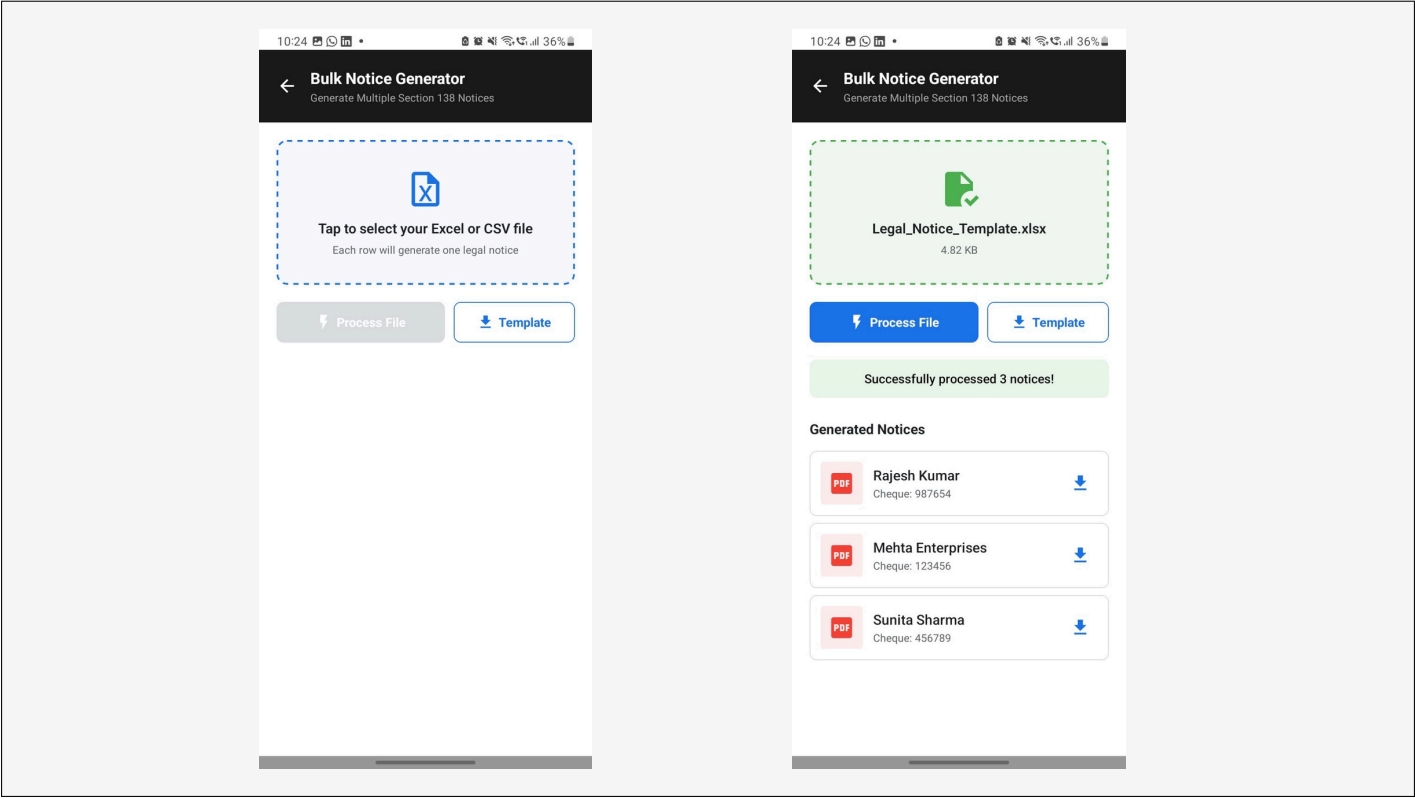
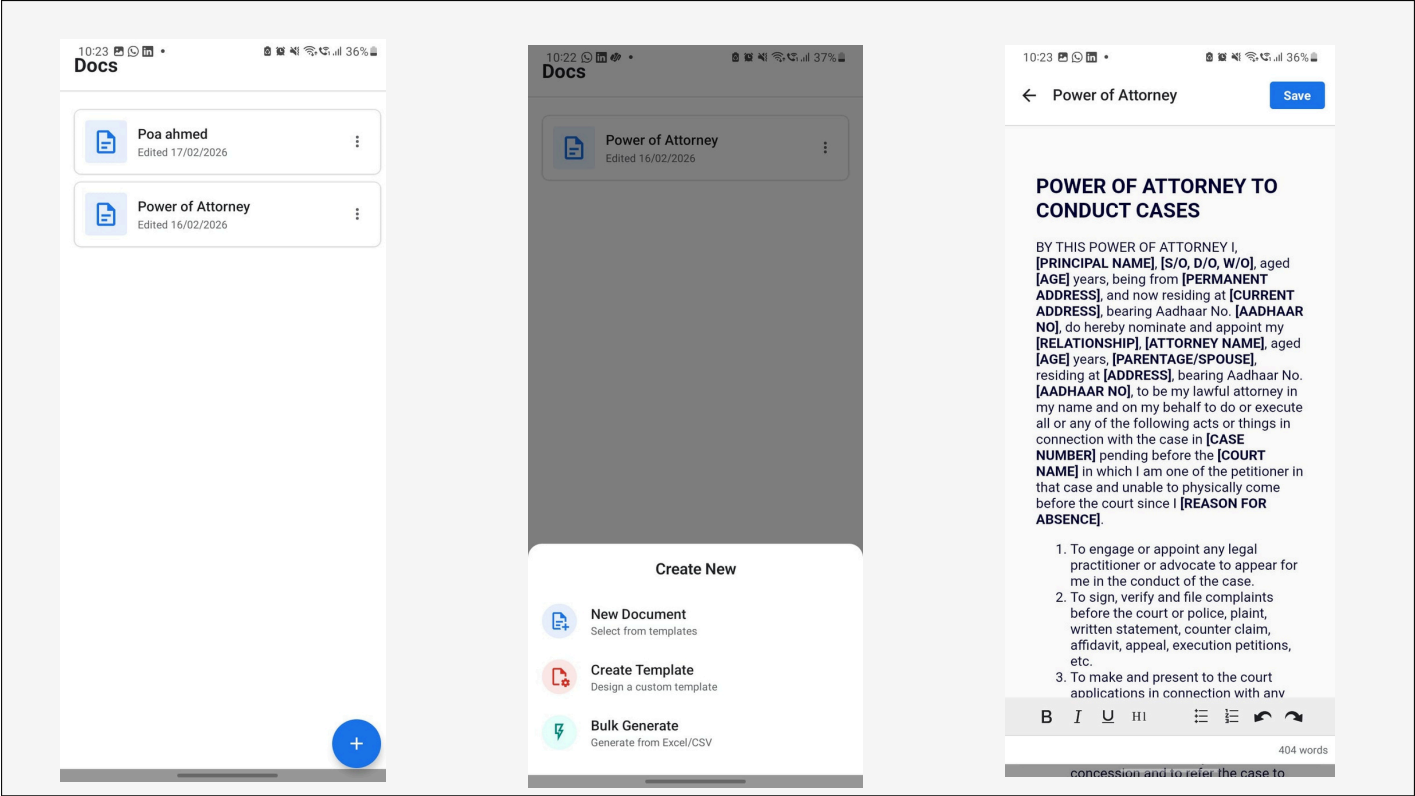
Full System Architecture



Project Output









NyayaSahayi

Expert on BNS, BNSS, and BSA.

Ask a legal question



NyayaSahayi

ask questions about BNS, BNSS, and BSA.



punishment for hit and run



Under the Bharatiya Nyaya Sanhita (BNS), the punishment for "hit and run" is primarily addressed when the incident results in the **death** of a person and the driver escapes without reporting.

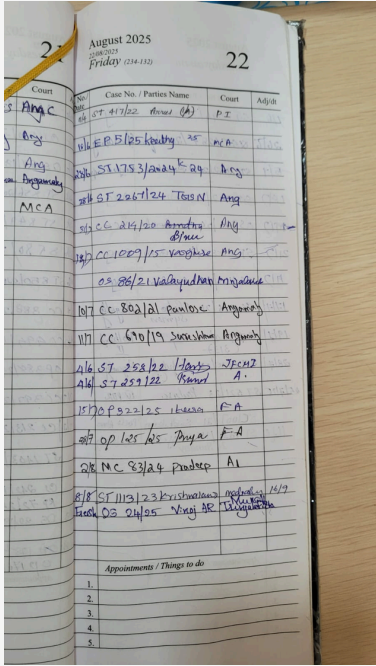
Here's a breakdown based on the provided sources:

1. Hit and Run causing Death (Specific Provision):

- **Section:** Bharatiya Nyaya Sanhita (BNS), Section 106(2)
- **Offence:** This section specifically deals with a situation where a person causes the **death** of any other person by **rash and negligent driving of a vehicle** (not amounting to culpable homicide), and **escapes without reporting it to a police officer or a Magistrate** soon after the incident

Ask a legal question (e.g., 'Punishment for hit and run')...





127.0.0.1:5000

☆

Lawyer Diary Text Extractor

Upload your diary page • Extract cases • Add to Google Calendar instantly

Choose Diary Image

Extract Text

Powered by AI • Secure & Private • Made for Lawyers

127.0.0.1:5000

☆

☒

CCM 690/19

Sureshkumar

Angamaly

11/9

☒

ST 258/22

Harts

JFCM 1

A

☒

ST 259/22

Sunil

A.

☒

OP 522/25

Ikeena

F.A

☒

OP 125/25

Priya

F.A

☒

MC 83/24

Pradeep

AI

☒

ST 1113/23

Krishnalamb

Medrabaly

16/9

☒

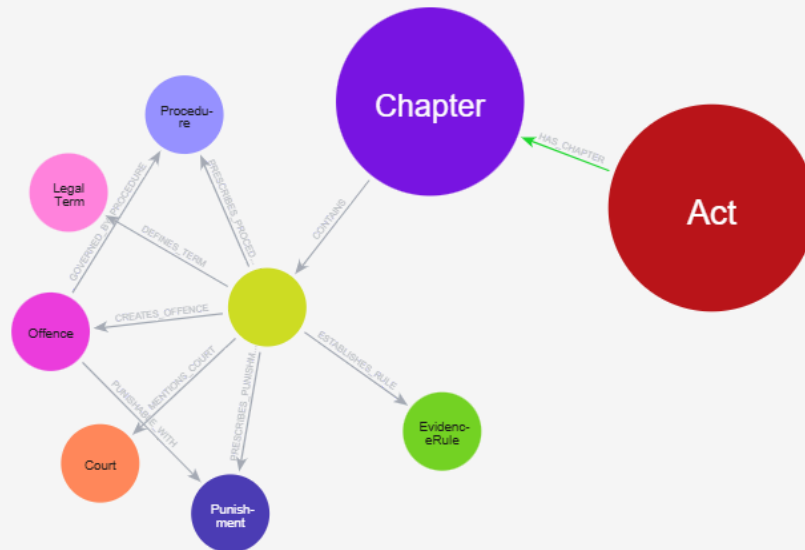
OS 24/25

Vinoj AR

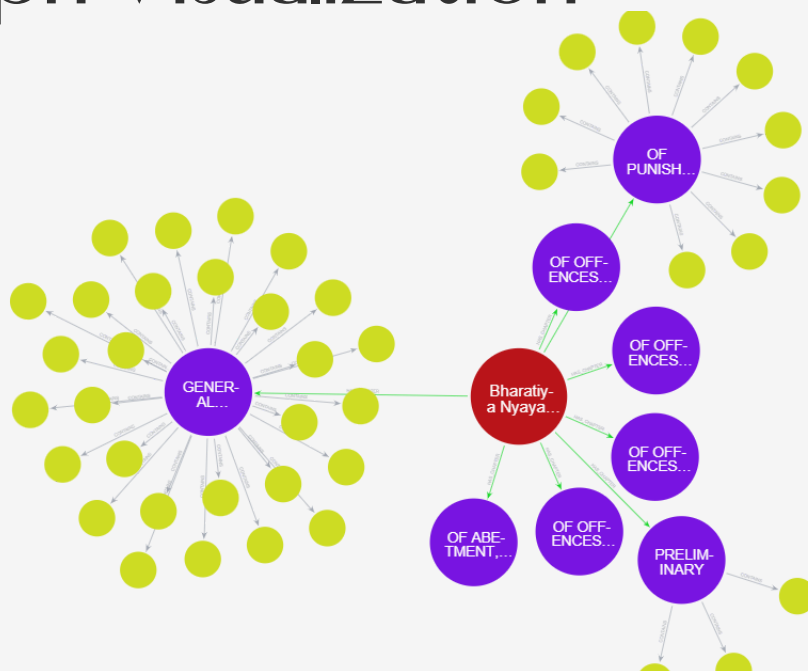
Triyagalala

Add Selected to Calendar

Graph Schema



Graph Visualization



Conclusion

This **RAG based legal assistant** streamlines a lawyer's workflow by integrating **intelligent automation into daily legal tasks**. With features such as an **AI Call Assistant** for managing calls, a **RAG-powered Legal Chatbot** for accurate query resolution, automated **diary-to-calendar syncing** via OCR, and a **smart notification system** for court dates, the system **reduces manual effort, minimizes missed deadlines, and enhances client communication**. Leveraging advanced AI algorithms and legal-domain retrieval methods, it delivers reliable, **context-aware** support tailored to the needs of legal professionals in the Indian context.



Thank You.

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

1. **Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions for complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
3. **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems, components or processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of

experiments, modelling, analysis and interpretation of data to provide valid conclusions.

5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, recognizing their limitations to solve complex engineering problems.
6. **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for their impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws.
8. **Individual and Collaborative Team Work:** Function effectively as an individual, and as a member or leader in diverse and multidisciplinary teams.
9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, and make effective presentations considering cultural, language and learning differences.
10. **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects in multidisciplinary environments.
11. **Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning, ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES:

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

CO-PO AND CO-PSO Mapping

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

Table 5.1: CO-PO-PSO Mapping

JUSTIFICATIONS FOR CO-PO MAPPING

MAPPING	LOW/ MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO1- PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/ CS822U.CO1- PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1- PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1- PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1- PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1- PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1- PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1- PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1- PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1- PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.

MAPPING	LOW/ MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO1- PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.
101003/ CS822U.CO1- PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1- PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2- PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.
101003/ CS822U.CO2- PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2- PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2- PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2- PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2- PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/ CS822U.CO2- PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2- PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2- PO10	Low	Requires basic planning and coordination during development tasks.

MAPPING	LOW/ MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO2- PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2- PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2- PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3- PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3- PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3- PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3- PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3- PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4- PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4- PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4- PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4- PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4- PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.

MAPPING	LOW/ MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO4- PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4- PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4- PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4- PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/ CS822U.CO4- PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4- PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5- PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5- PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5- PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5- PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5- PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5- PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5- PO11	High	Strongly promotes independent learning and exploration of emerging technologies.

MAPPING	LOW/ MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO5- PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/ CS822U.CO6- PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6- PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6- PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6- PO8	Medium	Communication supports collaboration within team environments.
101003/ CS822U.CO6- PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6- PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6- PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6- PSO3	High	Communication of innovative computer science solutions and research outcomes.